



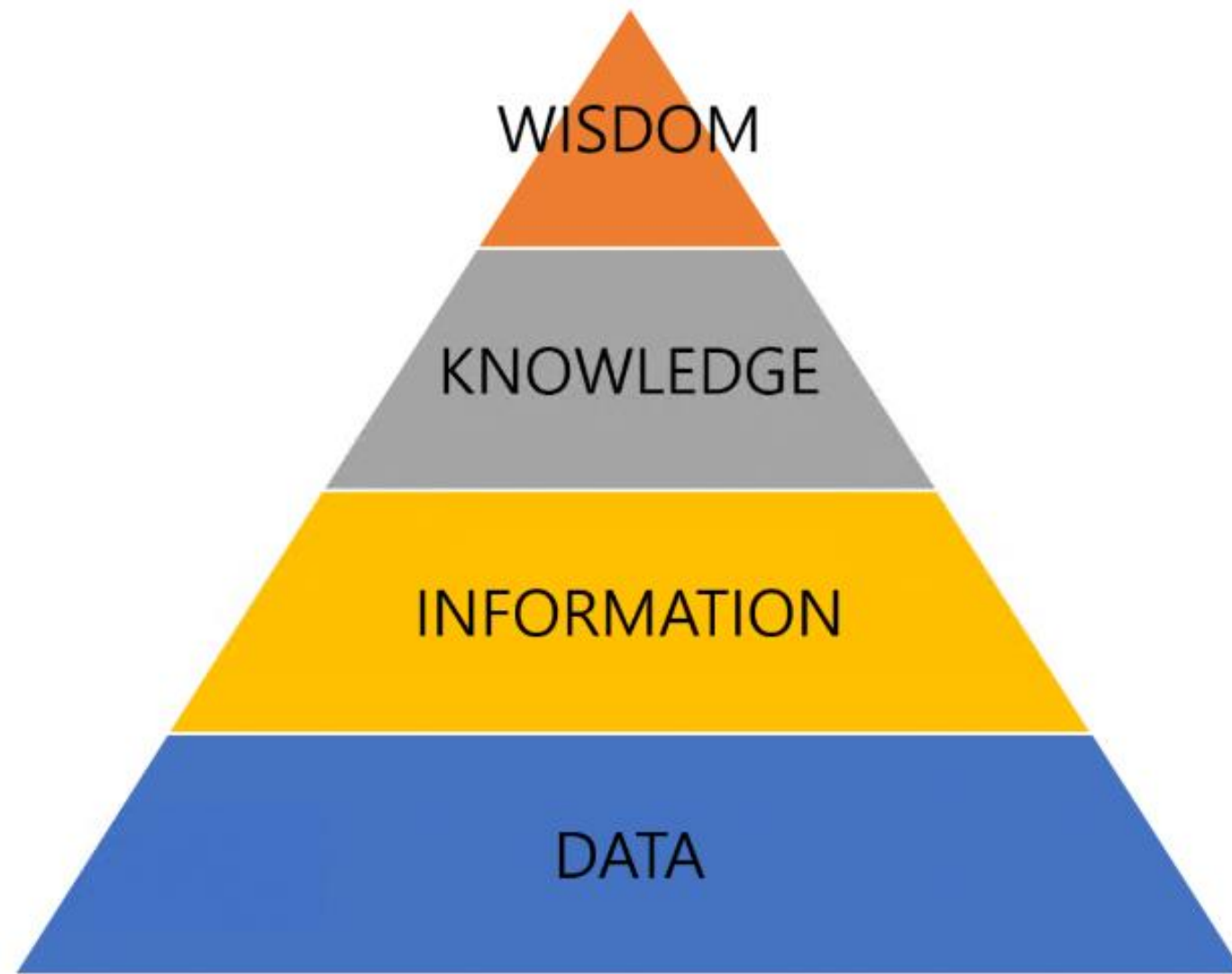
관계형 DATABASE의 이해와 활용

Data

Data와 Datum은 라틴어 어원에서 유래.

- **Datum (단수형):**
단 하나의 가공되지 않은 사실이나 측정값, 정보의 최소 단위를 의미.
- **Data (복수형):**
Datum이 여러 개 모인 자료의 집합체(데이터)를 의미.

DIKW 피라미드



- **Wisdom(지혜)** : "왜 그래야 하는가?" → 장기적 가치 판단과 결단
- **Knowledge (지식)** : "어떻게 해야 하는가?" → 패턴 발견과 행동 지침
- **Information (정보)** : "무슨 일이 일어났는가?" → 맥락이 부여된 데이터
- **Data (자료)** : 가공되지 않은 낱것의 수치와 사실

DataBase 정의

데이터베이스(DB)는 여러 사람이나 응용 프로그램이 공유하고 동시에 접근할 수 있도록 체계적으로 통합·저장한 데이터의 집합

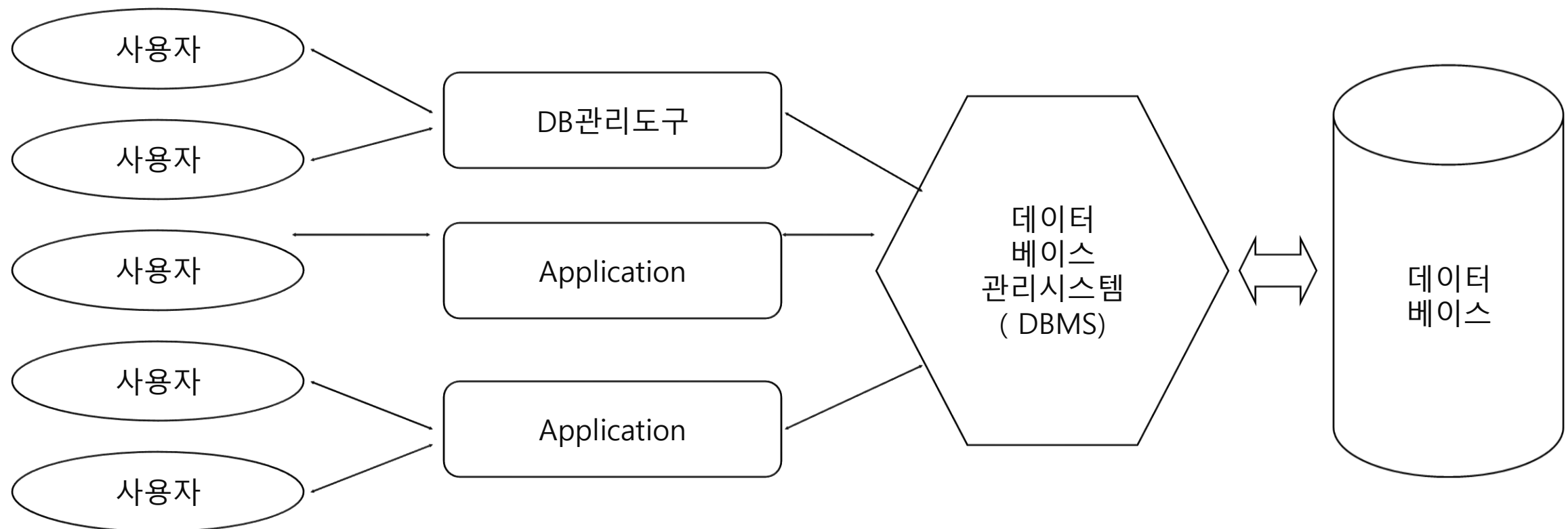
- 연관성 있는 **Data**의 집합체
- **DBMS(Database Management System)**을 통해서 관리
- **DBMS**를 통해서 **Data**의 추가, 수정, 삭제가 가능
- **DBMS**를 통해서 **Data**의 정렬 및 조회가 가능

DataBase 특징

- **실시간 접근성 (Real-Time Accessibility):** 사용자의 데이터 요구에 즉각적으로 응답하여 실시간으로 처리.
- **계속적인 변화 (Continuous Evolution):** 데이터의 삽입, 삭제, 수정을 통해 항상 최신의 정확한 상태를 유지.
- **동시 공유 (Concurrent Sharing):** 여러 사용자가 동시에 동일한 데이터를 서로 다른 목적으로 사용할 수 있어야 함.
- **내용 기반 참조 (Content-Based Reference):** 데이터의 저장 위치나 주소가 아닌, 데이터가 가진 '값(내용)'을 통해 데이터를 참조하고 찾아야 함.

DBMS 개요

- 방대한 양의 데이터를 편리하게 관리하고 효율적으로 저장하고 검색할 수 있는 환경을 제공해주는 시스템
소프트웨어로 DBMS의 가장 일반적인 형태가 RDBMS
- 데이터를 입력, 수정, 삭제 등의 관리 및 Database를 관리하는 프로그램



DBMS 역사

- 1960년대: 계층형 DBMS
- 1970년대: 망형 DBMS
- 1970년 : 에드거 F. 커드가 제안한 데이터 관계형 모델에 기초하는
디지털 데이터베이스 소개
- 1979년 : 최초의 상용 RDBMS인 Oracle 출연
- 1983년 : IBM DB2 발표
- 1986년 : ANSI(American National Standards Institute) SQL 표준 제정
- 1990년 : OLTP 사용
- 1994년 : 분산 DB
- 1997년 : ORDBMS, OODBMS

데이터베이스 용어

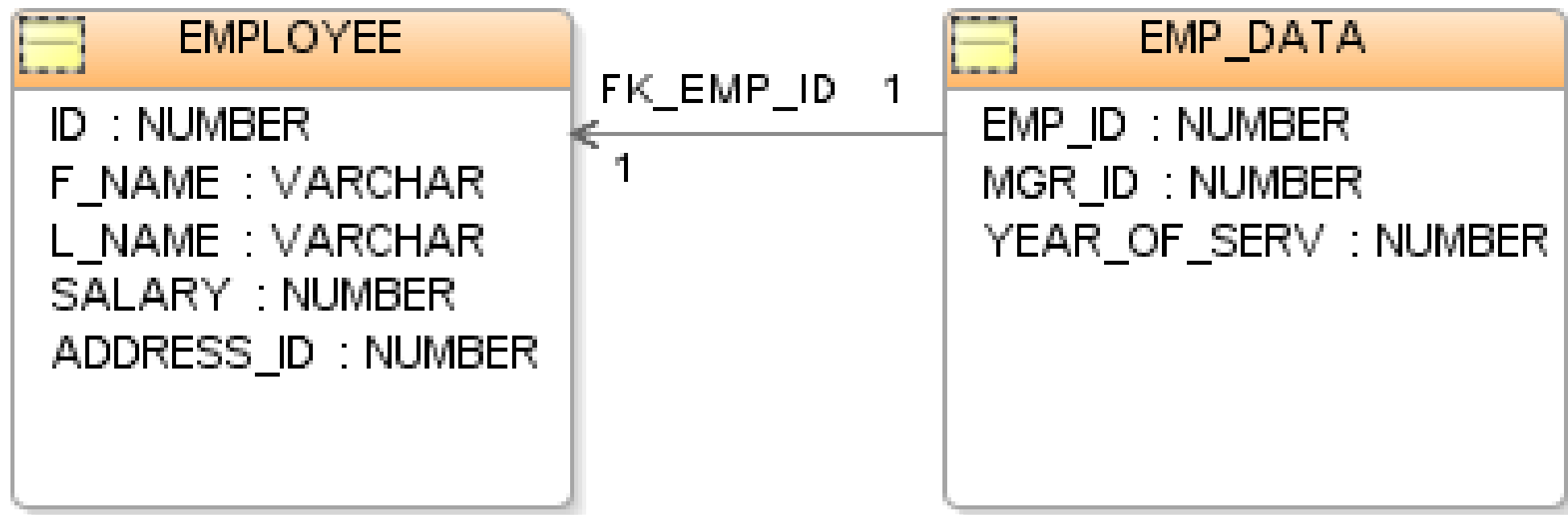
학생정보 테이블(릴레이션)

학번을 나타내는 컬럼, 속성, 필드					성명을 나타내는 컬럼, 속성, 필드					학과를 나타내는 컬럼, 속성, 필드				
학번	성명	학과	학년	성별	테이블의 스키마									
1001	김남준	컴퓨터공학	1	남	1001에 대한 tuple, row, record									
1002	김석진	정보통신공학	2	남	1002에 대한 tuple, row, record									
1003	민윤기	영문학	3	남	각각의 row가 학생테이블의 인스턴스									
1004	박지민	경영학	4	남										

- 테이블: 연관된 데이터들이 행과 열로 구성된 정보모음

관계(Relationship)

- 업무적인 연관선.
- 두 Table 사이에 존재.
- 관계를 맺고 있는 두 Table 중에 하나는 부모 Table 나머지 하나는 자식 Table.
- 부모 Table의 기본키는 자식 Table 의 외래키로 전이된다.



자동차 세일즈맨의 고객정보

고객번호	이름	주민번호	연락처	메일주소	...	...	차량번호	차종	차명	색깔	연식	판매일
1000	홍길동	700503-1000000	010-0000-0000	h@daum.net			23하4567	SUV	GV90	그린	2026	2026-12-21
1001	Alan Turing	030913-3000000	011-1111-1111	at@daum.net			108나7890	세단	현대 쏘나타	그린	2021	2021-10-12
1002	김연아	721007-2000000	010-2222-2222	k@naver.com			32다9876	SUV	싼타페	그레이	2026	2026-02-24
1003	Irving John Good	730504-1000000	016-3333-3333	j@outlook.com			56라1234	전기차	테슬라 모델3	블랙	2025	2025-02-28
1004	박지성	700528-1000000	010-4312-8471	p@naver.com			78마5678	세단	에쿠스	화이트	2020	2020-01-24
1005	Florence Nightingale	981211-2000000	010-4589-6439	fn@naver.com			01바2345	전기차	아이오닉7	그레이	2026	2026-11-25
1006	Ada Lovelace	050802-4000000	010-4727-7850	al@gmail.com			105사6789	전기차	EV9	블루	2024	2024-02-20
1007	유재석	780502-1000000	010-7845-4190	s@gmail.com			77아1234	세단	그랜저	실버	2023	2023-06-27
1008	이도	100421-3000000	010-1114-3924	d@outlook.com			108자5678	해치백	아반떼	블루	2024	2024-02-08
1009	이지은	780102-2000000	010-8206-5957	u@outlook.com			99가4321	세단	현대 쏘나타	레드	2021	2021-06-05

Relation Ship(관계)

고객번호	이름	주민번호	연락처	메일주소	...	...
1000	홍길동	700503-1000000	010-0000-0000	h@daum.net		
1001	Alan Turing	030913-3000000	011-1111-1111	at@daum.net		
1002	김연아	721007-2000000	010-2222-2222	k@naver.com		
1003	Irving John Good	730504-1000000	016-3333-3333	j@outlook.com		
1004	박지성	700528-1000000	010-4312-8471	p@naver.com		
1005	Florence Nightingale	981211-2000000	010-4589-6439	fn@naver.com		
1006	Ada Lovelace	050802-4000000	010-4727-7850	al@gmail.com		
1007	유재석	780502-1000000	010-7845-4190	s@gmail.com		
1008	이도	100421-3000000	010-1114-3924	d@outlook.com		
1009	이지은	780102-2000000	010-8206-5957	u@outlook.com		

차량번호	차종	차명	색깔	연식	판매일	고객번호
23하4567	SUV	GV90	그린	2026	2026-12-21	1000
108나7890	세단	현대 쏘나타	그린	2021	2021-10-12	1001
32다9876	SUV	싼타페	그레이	2026	2026-02-24	1002
78마5678	세단	에쿠스	화이트	2020	2020-01-24	1004
77아1234	세단	그랜저	실버	2023	2023-06-27	1007
108자5678	해치백	아반떼	블루	2024	2024-02-08	1008

Relation Database Model

- RDB는 Relation Database Model을 구현하며,
- E.F. Code 박사가 정의한 관계형 데이터베이스 규칙을 따름.

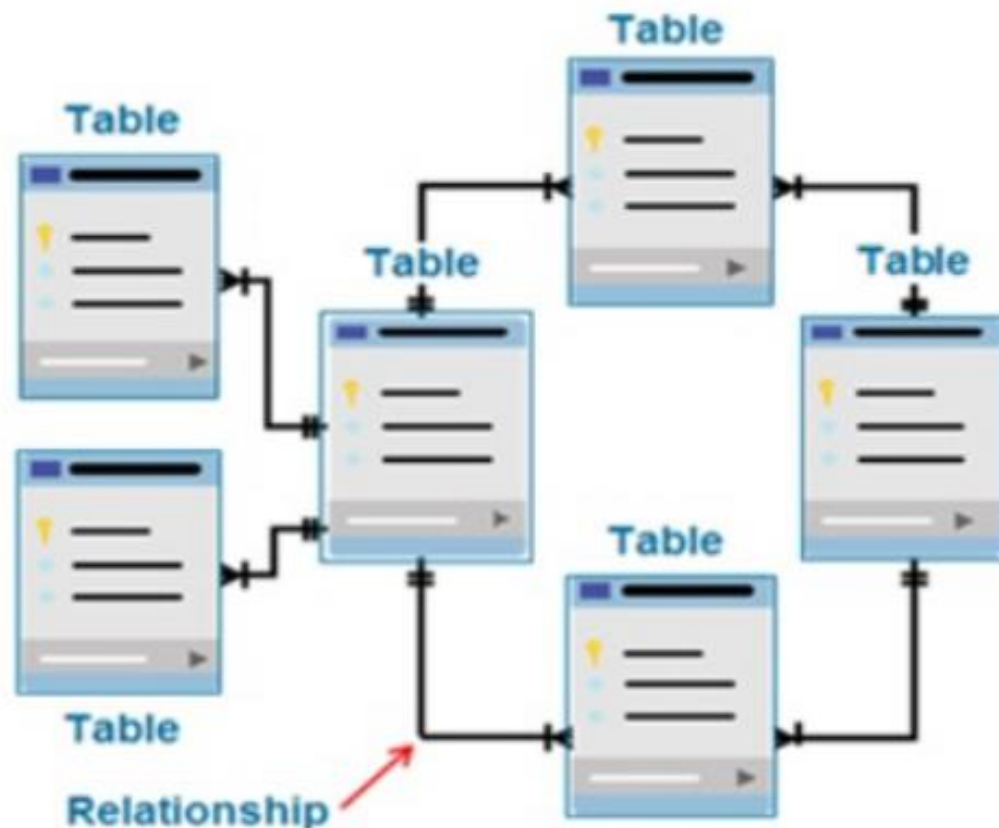


Table = Relation = Entity = Concept = Object
PK - Primary Key And FK - Foreign Key

StuID	StuName	StuAge	StuClass	StuSection
1001	Alex	15	10	B
1002	Maria	14	11	A
1003	Maya	14	9	A
1004	Bob	16	11	C
1005	Newton	14	10	D
1006	Sanjay	15	10	B

A **Table** Represents a Database **Entity**

Table **Row** is referred as **Records** Or **Tuple**

A Table **Column** Represents an **Attribute**

Relation Database Model

- RDB는 상호 연관된 Table들의 그룹으로 논리적으로 구성.
- 각 Table은 여러 행과 열을 가질 수 있음.
- Table은 Record들의 모음이며, 데이터베이스 Entity(엔티티) 를 나타냄.

- RDB는 상호 연관된 Table들의 그룹으로 논리적으로 구성.
- 각 Table은 여러 행과 열을 가질 수 있음.
- Table은 Record들의 모음이며, 데이터베이스 Entity(엔티티) 를 나타냄.
- Table의 각 Row(행)은 여러 데이터 Fields(필드)로 구성된 Record(Tuple/Row) 를 나타냄.
- Table의 Column은 Entity의 Attributes(속성) 을 나타냄.

StuID	StuName	StuAge	StuClass	StuSection
1001	Alex	15	10	B
1002	Maria	14	11	A
1003	Maya	14	9	A
1004	Bob	16	11	C
1005	Newton	14	10	D
1006	Sanjay	15	10	B

A Table Represents a Database Entity

Table Row is referred as Records Or Tuple

A Table Column Represents an Attribute

Relation Database Model

- Row 와 Column이 교차하는 부분을 Fields(필드) 라고 하며, 실제로 데이터가 저장되는 영역.
- Entity(엔티티) 는 데이터베이스에 표현되어야 하는 실제 세계의 객체 또는 비즈니스 개념.

- 데이터베이스 엔티티는 2차원 테이블로 표현 됨.
- 다양한 데이터 요소 간에 존재하는 함수적 종속성 에 따라, 두 테이블(엔티티) 간의 관계 는 데이터베이스 키 를 사용하여 설정됨.
- 데이터베이스 키 는 기본 키 와 외래 키 를 사용하여 테이블 간의 연관 관계를 생성하도록 정의됨.

StuID	StuName	StuAge	StuClass	StuSection
1001	Alex	15	10	B
1002	Maria	14	11	A
1003	Maya	14	9	A
1004	Bob	16	11	C
1005	Newton	14	10	D
1006	Sanjay	15	10	B

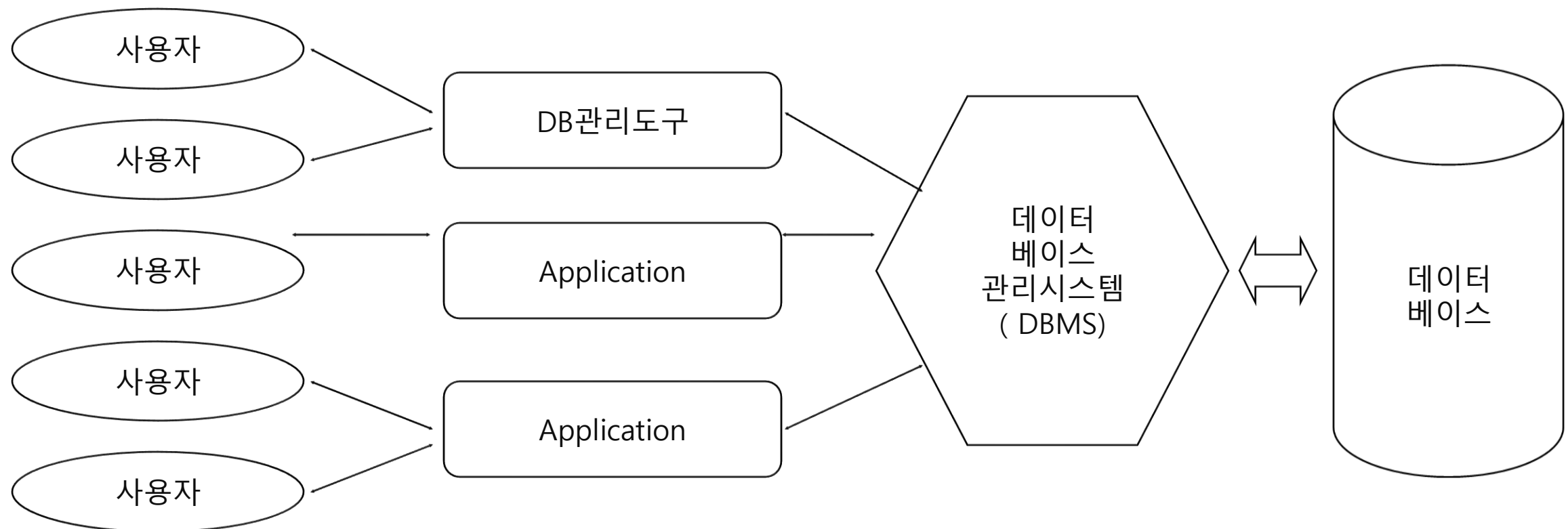
A Table Represents a Database Entity

Table Row is referred as Records Or Tuple

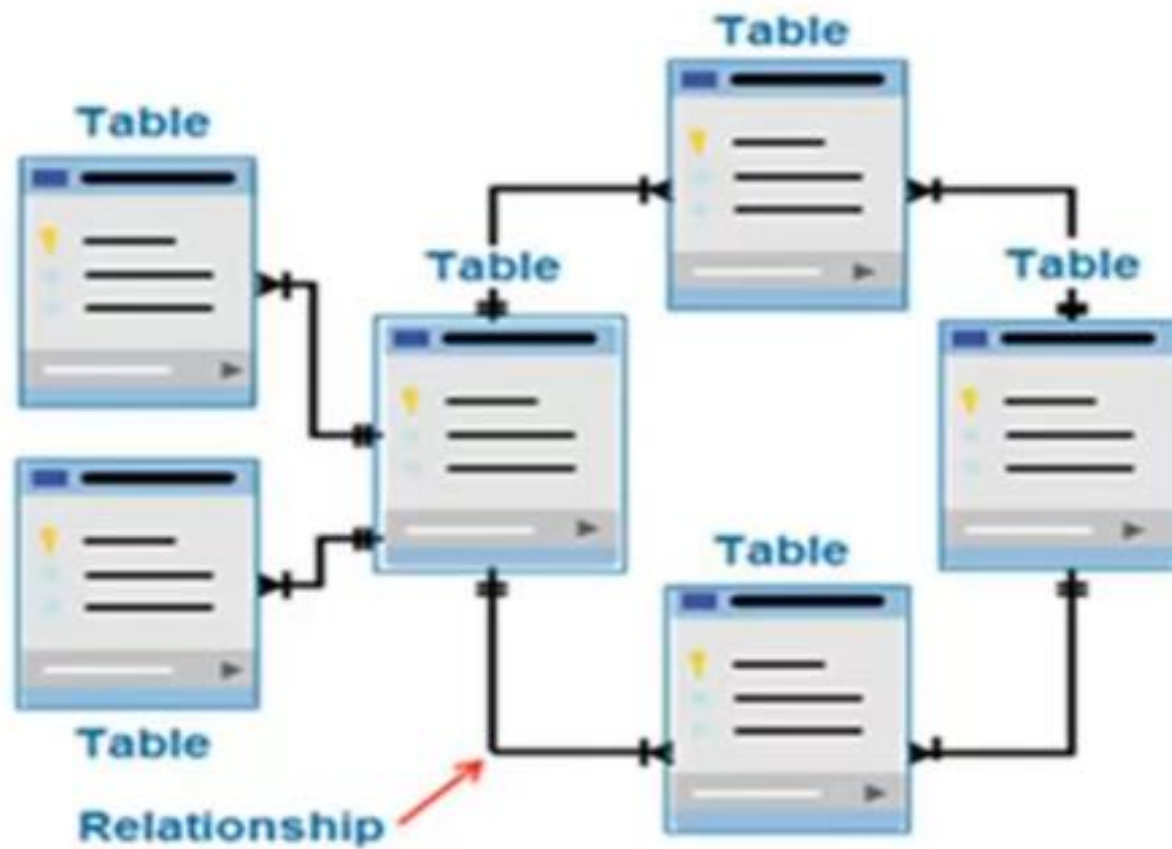
A Table Column Represents an Attribute

Database Management System

- 방대한 양의 데이터를 편리하게 관리하고 효율적으로 저장하고 검색할 수 있는 환경을 제공하는 시스템 소프트웨어로 **DBMS의 가장 일반적인 형태가 RDBMS**
- 데이터를 입력, 수정, 삭제 등의 관리 및 Database를 관리하는 프로그램
- NoSQL 및 객체 지향 데이터베이스와 같은 새로운 Database도 등장



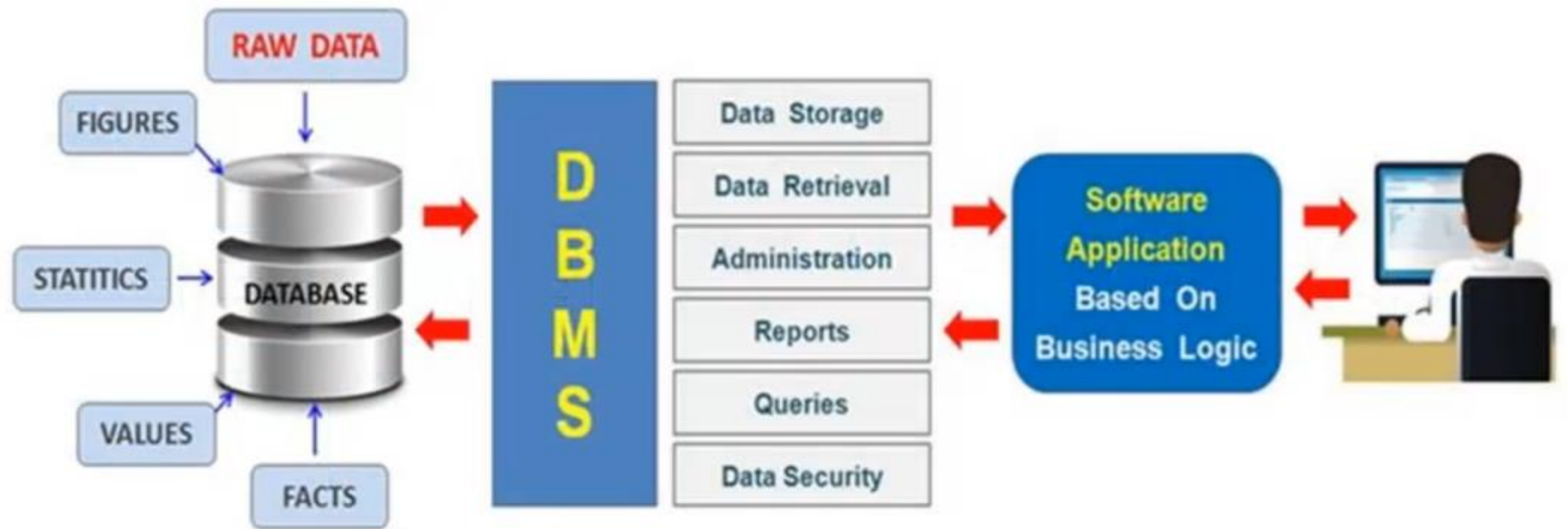
RDBMS



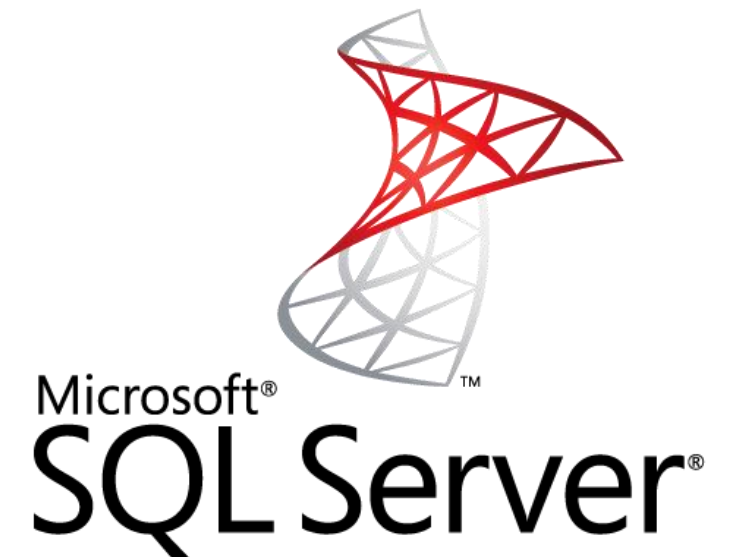
RDBMS

Relational Databases

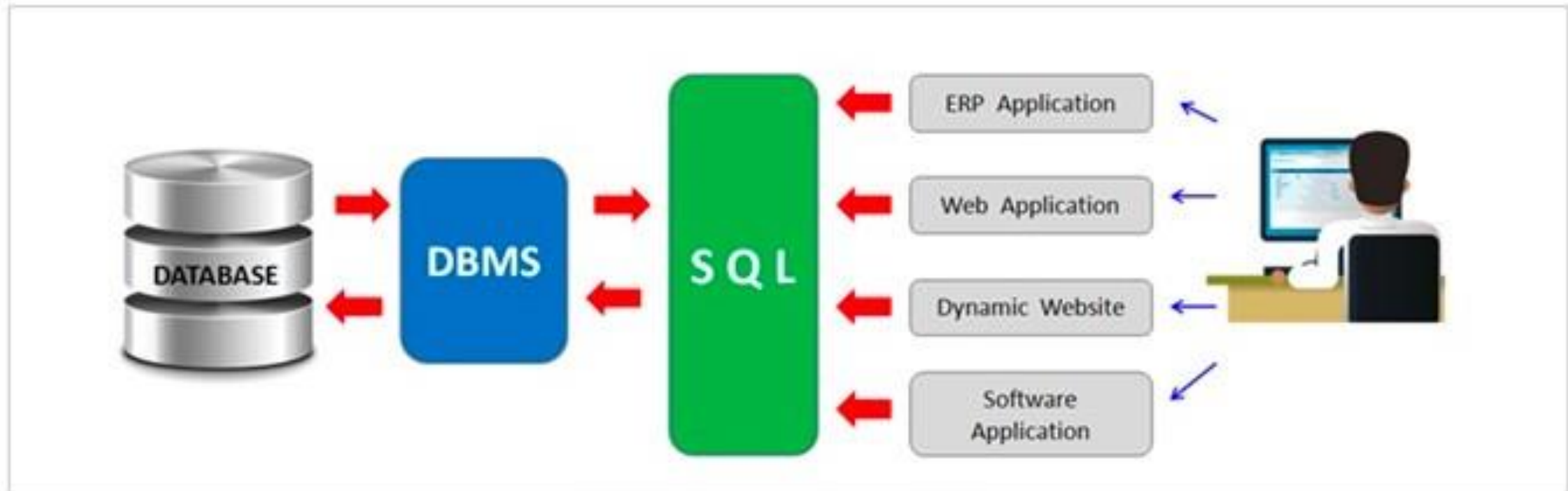
RDBMS



다양한 Database Management System

The Oracle logo consists of the word "ORACLE" in a white, sans-serif font, with a registered trademark symbol (®) to the upper right of the "E". It is set against a solid red rectangular background.The IBM logo features the letters "IBM" in a white, bold, sans-serif font. The letters are composed of horizontal stripes, with the "I" and "M" having more stripes than the "B". It is set against a solid black rectangular background.The DB2 logo features the letters "DB2" in a white, bold, sans-serif font, set against a solid green rectangular background.The Microsoft SQL Server logo features a stylized, three-dimensional wireframe sphere in red and grey, resembling a globe or a network. Below it, the text "Microsoft® SQL Server®" is written in a black, sans-serif font.The PostgreSQL logo features a blue elephant head icon. Below it, the text "PostgreSQL" is written in a blue, sans-serif font.The SQLite logo features a blue square icon with a white feather inside. To the right of the icon, the text "SQLite" is written in a blue, sans-serif font.The Sybase logo features the word "SYBASE" in a large, bold, serif font, with a registered trademark symbol (®) to the upper right of the "E".The MySQL logo features a blue dolphin icon. Below it, the text "MySQL" is written in a blue, sans-serif font, with a registered trademark symbol (®) to the upper right of the "L".The MariaDB logo features a blue seal icon. Below it, the text "MariaDB" is written in a blue, sans-serif font.

RDBMS



DBMS COMPONENTS

SQL - Structured Query Language. **DBMS** - Database Management System.

Database - Organized Collection Of Interrelated Data.

www.learncomputerscienceonline.com

SQL(Structured Query Language)

관계형 데이터베이스 관리 시스템(RDBMS)에서 데이터를 저장, 수정, 삭제하고 검색하기 위해 사용하는 특수 목적의 프로그래밍 언어

- 다양한 데이터베이스 사용을 위해 표준화
- 인간의 언어와 흡사(영어)
- **SQL-86 : ANSI에 의해서 최초의 표준화**
- **SQL:2023 : SQL-86의 6번째 edition**

SQL(Structured Query Language)

- **데이터 정의 언어 (DDL: Data Definition Language)**

- 데이터베이스 구조(테이블, 인덱스 등)를 생성, 수정, 삭제하는 명령어.
- CREATE (생성), ALTER (수정), DROP (삭제)

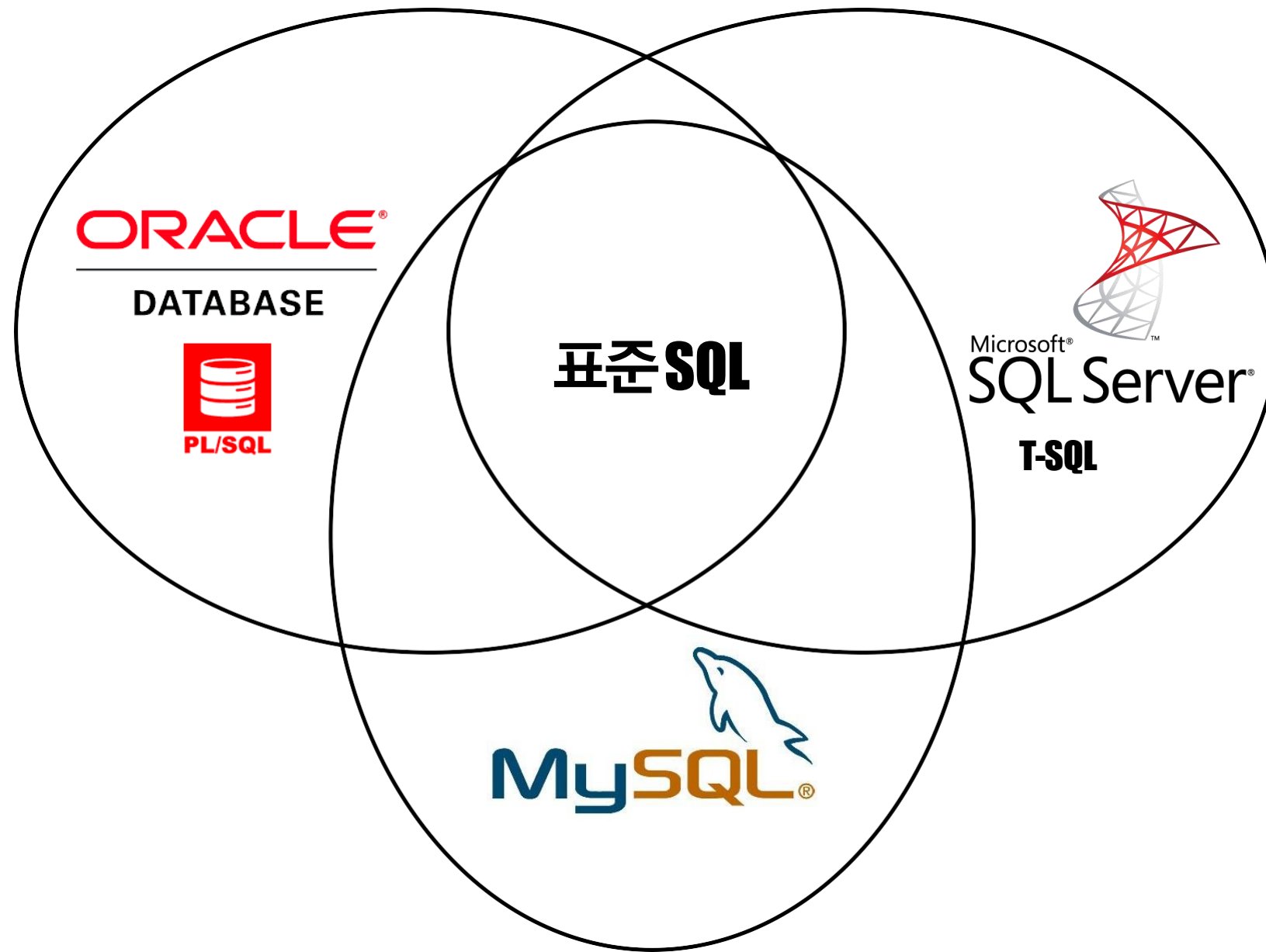
- **데이터 조작 언어 (DML: Data Manipulation Language)**

- 실제 데이터를 검색하거나 입력, 수정, 삭제할 때 사용하는 명령어.
- SELECT (조회), INSERT (입력), UPDATE (수정), DELETE (삭제)

- **데이터 제어 언어 (DCL: Data Control Language)**

- 데이터베이스에 대한 권한을 부여하거나 트랜잭션(작업 단위)을 관리하는 명령어.
- GRANT (권한 부여), REVOKE (권한 회수)

표준 SQL



데이터베이스 용어

학생정보 테이블(릴레이션)

학번	성명	학과	학년	성별
1001	김남준	컴퓨터공학	1	남
1002	김석진	정보통신공학	2	남
1003	민윤기	영문학	3	남
1004	박지민	경영학	4	남

학번을 나타내는 컬럼, 속성, 필드

성명을 나타내는 컬럼, 속성, 필드

학과를 나타내는 컬럼, 속성, 필드

테이블의 스키마

1001에 대한 tuple, row, record

1002에 대한 tuple, row, record

각각의 row가 학생테이블의 인스턴스

테이블: 연관된 데이터들이 행과 열로 구성된 정보모음

컬럼, 속성

- 컬럼명의 단일테이블에서 유일해야함
- 컬럼은 스키마에 정의된 도메인 내의 값만 가질 수 있음
- 컬럼은 단일 값을 가짐 (**ex**) 정보통신, 컴공 **x**
- 컬럼의 순서는 무관함

키(key)

- 테이블내의 인스턴스를 식별하기 위해 사용가능한 값을 **key**라고 함(후보키)
- 테이블 내의 키중에 선택해서 대표로 사용하는 키를 기본 키 (**Primary Key**)라고 함
- 일련번호 같은 데이터와 직접 연관없는 속성을 만들어서 기본 키로 사용하는 경우가 있는데 이를 대리키라고 함
- 다른 테이블의 기본키를 참조하는 속성을 외래키(**Foreign Key**)라고함

기본키(Primary key)

- 중복되지않는 유일한 값(Unique)
- Null(값이 없음)을 가질 수 없음
- 테이블내에 하나의 기본키만 가질 수 있음
- 하나의 키로 인스턴스 식별을 할 수 없을때는 복수의 키를 기본 키로 사용하며 이를 복합키라고 함

데이터 무결성을 위한 제약 조건

구분	제약조건
기본키 제약	테이블내의 각 row를 식별하기 위한것으로 null 이나 중복된값을 허용하지 않는다.
외래키 제약	해당 속성은 참조되는 테이블의 속성 값들 중에 하나와 일치하거나 null만을 가질수 있다.
Unique 제약	테이블의 모든 row에 대해서 유일한 값을 가져야 한다.
Not Null 제약	Not Null로 설정된 속성은 Null을 가질 수 없다.
Check 제약	해당 속성에 저장 데이터의 값의 타입이나 범위가 지정된 조건을 벗어날 수 없다.

PostgreSQL

PostgreSQL 포스트그레스큐엘



```

vulcan@phosphor:~/dev/entire-line-topaz - $ sudo su postgres
postgres@vulcan:~/dev/entire-line-topaz /home/vulcan@phosphor:~$ psal
psal (17.5)
Type "help" for help.

postgres=# \copyright
PostgreSQL Database Management System
(formerly known as Postgres, then as Postgres95)

Portions Copyright (c) 1996-2004, PostgreSQL Global Development Group
Portions Copyright (c) 1994, The Regents of the University of California

Permission to use, copy, modify, and distribute this software and its
documentation for any purpose, without fee, and without a written agreement
is hereby granted, provided that the above copyright notice and this
paragraph and the following two paragraphs appear in all copies.

IN NO EVENT SHALL THE UNIVERSITY OF CALIFORNIA BE LIABLE TO ANY PARTY FOR
DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, INCLUDING
LOST PROFITS, ARISING OUT OF THE USE OF THIS SOFTWARE AND ITS
DOCUMENTATION, EVEN IF THE UNIVERSITY OF CALIFORNIA HAS BEEN ADVISED OF THE
POSSIBILITY OF SUCH DAMAGE.

THE UNIVERSITY OF CALIFORNIA SPECIFICALLY DISCLAIMS ANY WARRANTIES,
INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY
AND FITNESS FOR A PARTICULAR PURPOSE. THE SOFTWARE PROVIDES HEREUNDER IS
ON AN "AS IS" BASIS, AND THE UNIVERSITY OF CALIFORNIA HAS NO OBLIGATIONS TO
PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

postgres=# \du

```

Role name	List of roles	Attributes
postgres	Superuser, Create role, Create DB, Replication, Bypass RL	
vulcan@phosphor	Create DB	

오픈 소스 객체-관계형 데이터베이스 관리 시스템
(ORDBMS)

개발자	The PostgreSQL Global Development Group
발표일	1996년 7월 8일(29년 전)
안정화 버전	18.4 ^[1] / 2026년 5월 14일(9일 전)
저장소	git.postgresql.org/gitweb/?p=postgresql.git
프로그래밍 언어	C
운영 체제	크로스 플랫폼
종류	관계형 데이터베이스
웹사이트	www.postgresql.org

PostgreSQL

- **INGRES(Interactive Graphics and Retrieval System)**

- 1970년대 UC Berkeley의 마이클 스톤브레이커(Michael Stonebraker) 교수가 개발한 초기 **관계형 데이터베이스 관리 시스템(DBMS)**

- **Post-Ingres (Postgres)**

- 1986년부터 스톤브레이커 교수팀이 기존 Ingres 모델의 한계를 극복하기 위해 **객체 지향 개념을 추가**하여 개발한 차세대(Post) 프로젝트

- **PostgreSQL**

- 1995년 프로젝트에 **SQL 언어 지원이 추가**되며 이름이 **Postgres95로 변경**되었고,
- 이후 1996년에 관계형 데이터베이스의 전통을 잇는다는 의미에서 현재의 **PostgreSQL**로 최종 명명

PostgreSQL Server 설치

- PostgreSQL 공식사이트에서다운로드
 - <https://www.postgresql.org/download/>

PostgreSQL Version	Linux x86-64	Linux x86-32	Mac OS X	Windows x86-64	Windows x86-32
18.4	postgresql.org 	postgresql.org 			Not supported
17.10	postgresql.org 	postgresql.org 			Not supported
16.14	postgresql.org 	postgresql.org 			Not supported
15.18	postgresql.org 	postgresql.org 			Not supported
14.23	postgresql.org 	postgresql.org 			Not supported
13.23	postgresql.org 	postgresql.org 			Not supported

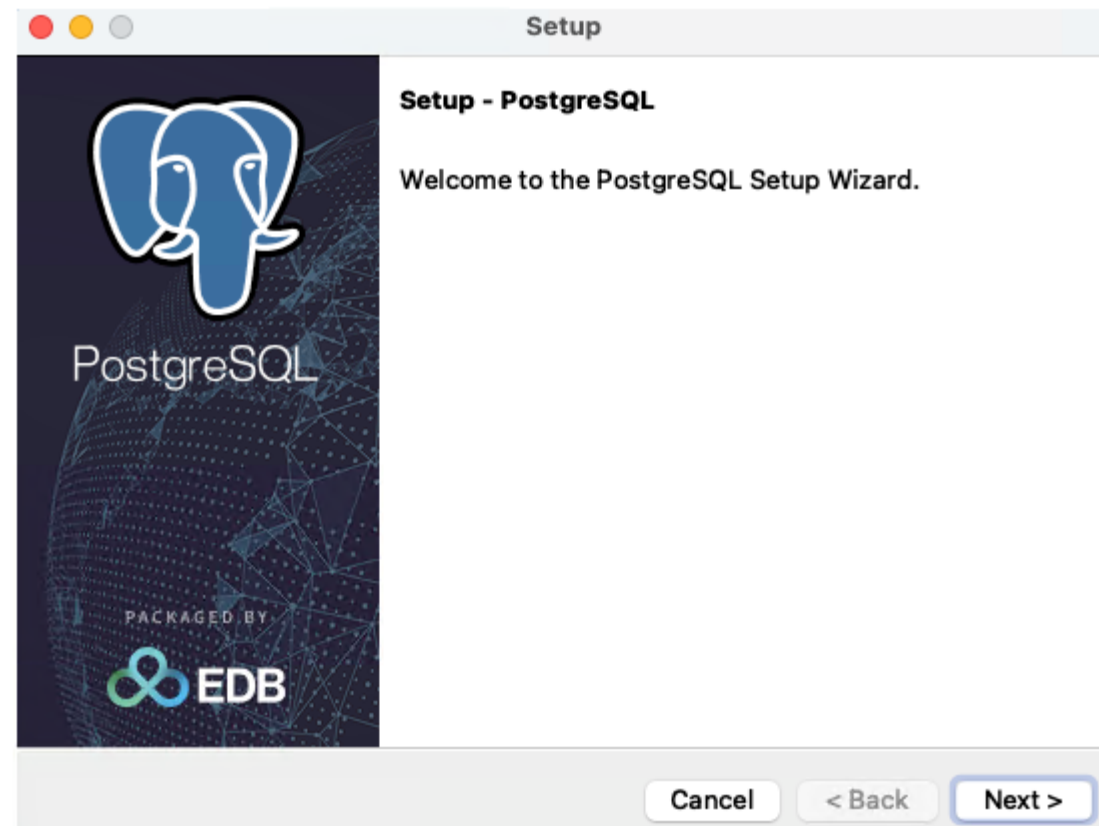
- 버전 : 18.x
- 원하는 OS 선택
 - Windows : Windows x86-64
 - MacOS : Mac OS X

PostgreSQL Server 설치

PostgreSQL 18.4-1



postgresql-18.4-1-osx.app



PostgreSQL Server 설치

Setup

Installation Directory

Please specify the directory where PostgreSQL will be installed.

Installation Directory

InstallBuilder

Cancel < Back Next >

Setup

Select Components

Select the components you want to install; clear the components you do not want to install. Click Next when you are ready to continue.

☒ PostgreSQL Server	PostgreSQL database server
☒ pgAdmin 4	
☒ Stack Builder	
☒ Command Line Tools	

InstallBuilder

Cancel < Back Next >

Setup

Data Directory

Please select a directory under which to store your data.

Data Directory

InstallBuilder

Cancel < Back Next >

Setup

Password

Please provide a password for the database superuser (postgres). A locked Unix user account (postgres) will be created if not present.

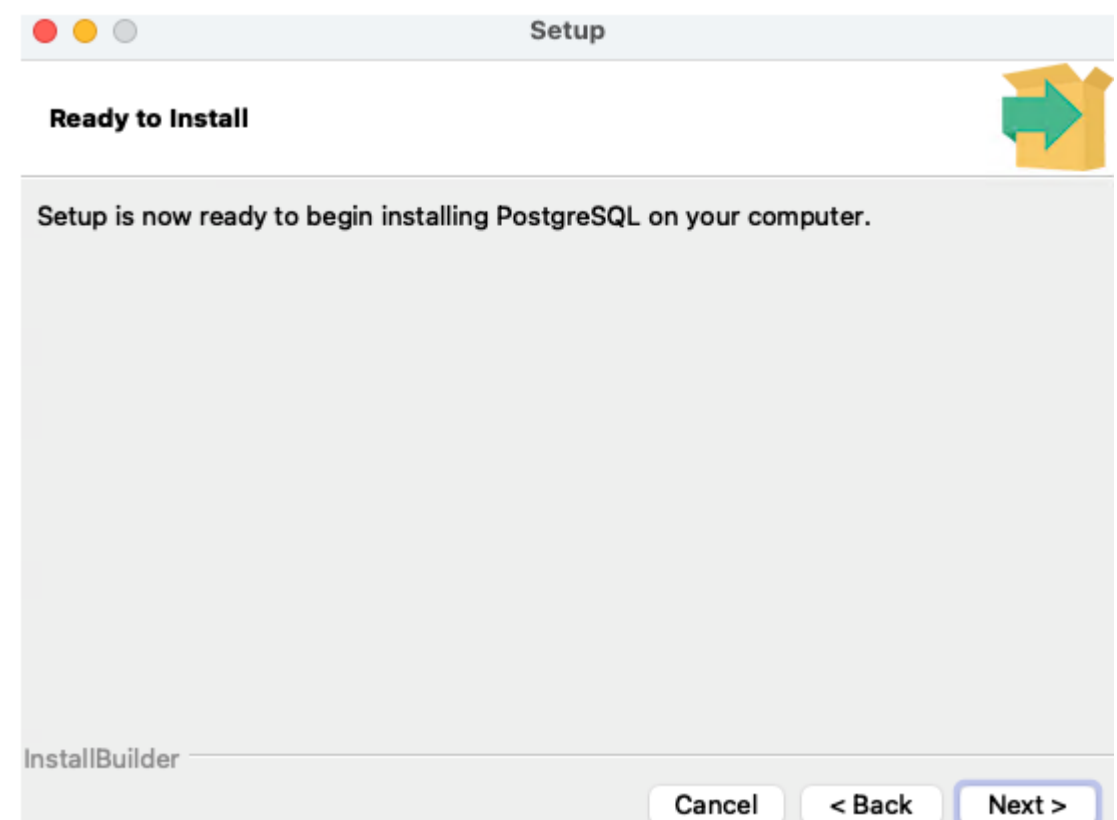
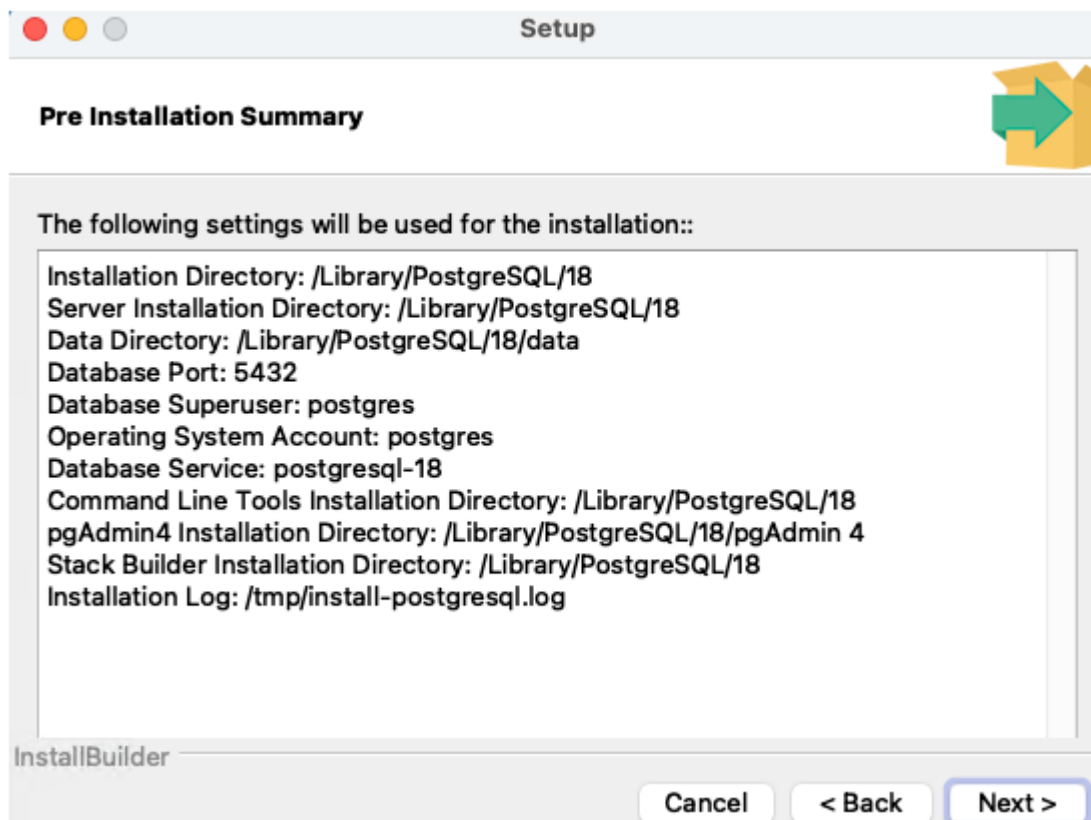
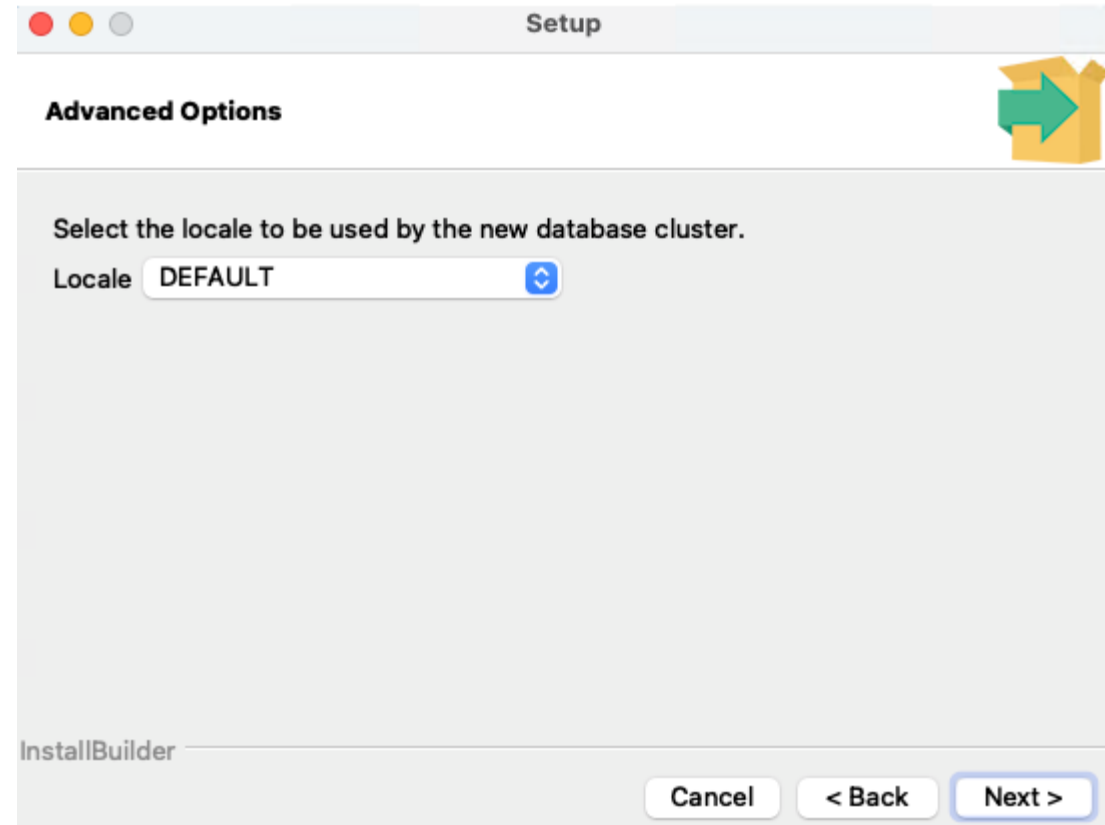
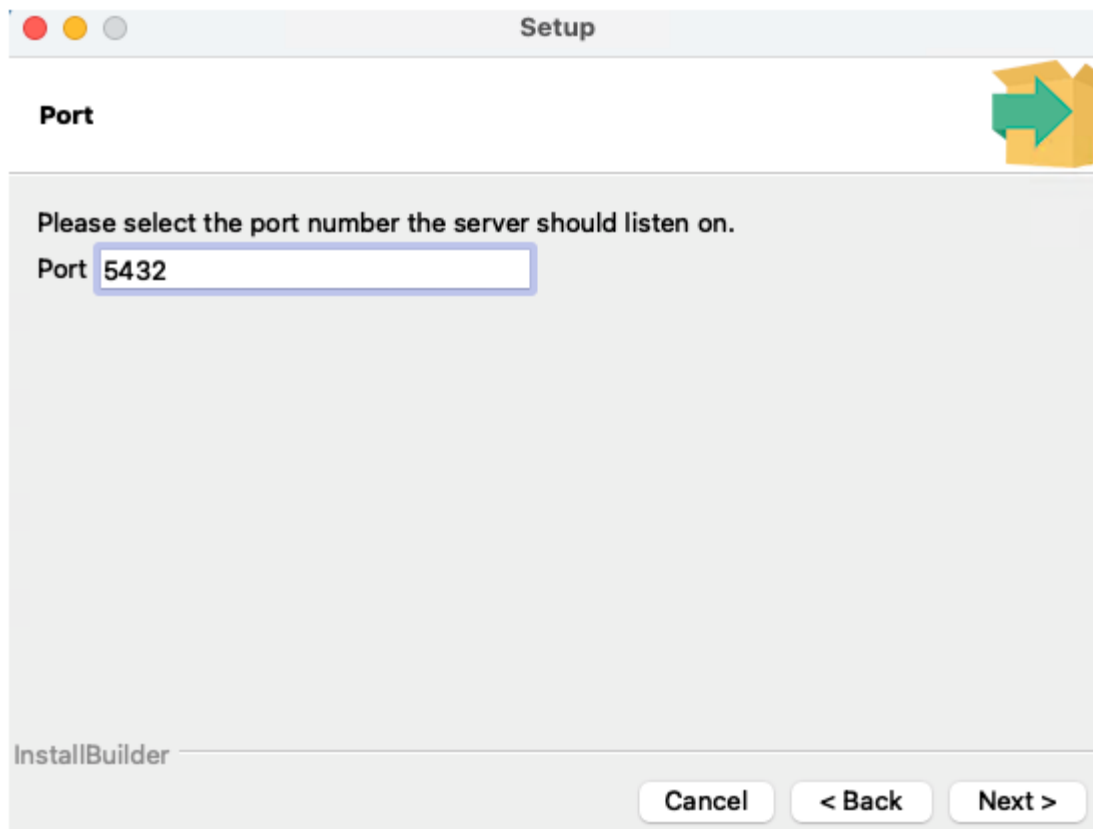
Password

Retype password

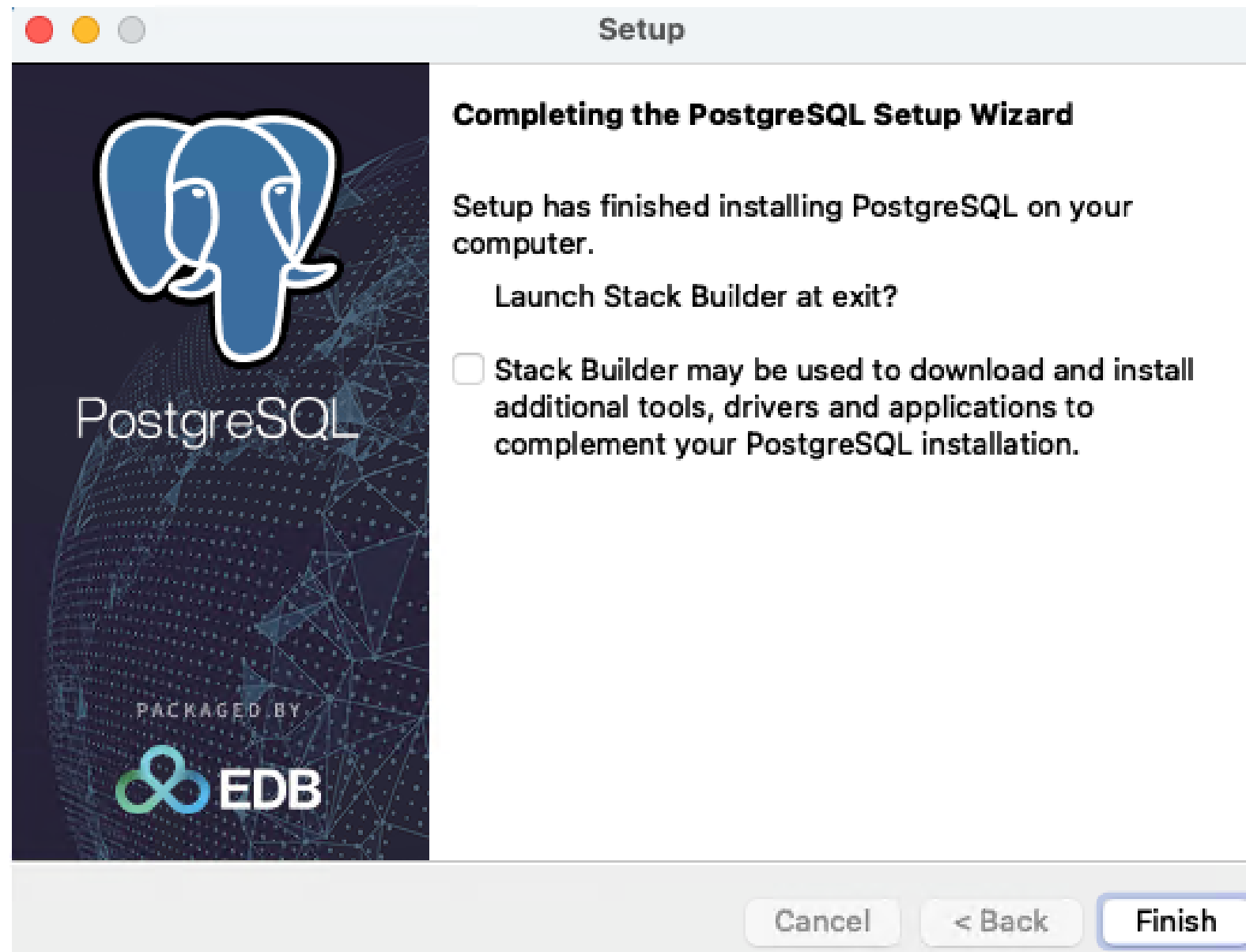
InstallBuilder

Cancel < Back Next >

PostgreSQL Server 설치



PostgreSQL Server 설치



PostgreSQL Client: SQL Shell



SQL Shell (psql)

- 사용자의 암호 입력항목이 나올 때 까지 계속 Enter 클릭
- 암호 입력

```
SQL Shell (psql)
Server [localhost]:
Database [postgres]:
Port [5432]:
Username [postgres]:
Password for user postgres:

psql (18.4)
WARNING: Console code page (437) differs from Windows code page (1252)
        8-bit characters might not work correctly. See psql reference
        page "Notes for Windows users" for details.
Type "help" for help.

postgres=# |
```

`psql -U postgres`

`psql -U postgres -d testdb -h localhost -p 5432`

PostgreSQL Client: SQL Shell

postgres=#

\encoding utf-8 <- 인코딩 에러시

select version(); <- 버전 확인

\? <- help

\l <- 데이터베이스 목록 확인

\du <- 사용자 목록 확인

\dt <- 테이블 목록 확인

\dv <- 뷰 목록 확인

SQL CRUD(DML)

- **Create : INSERT**
- **Retrieve(Read) : SELECT**
- **Update : UPDATE**
- **Delete : DELETE**

PostgreSQL Client: SQL Shell

```
postgres=#
```

```
\l  
create database testdb  
\c testdb
```

```
testdb=#
```

```
create table t1(tno integer, tname varchar(10));
```

```
\d  
\dt t1
```

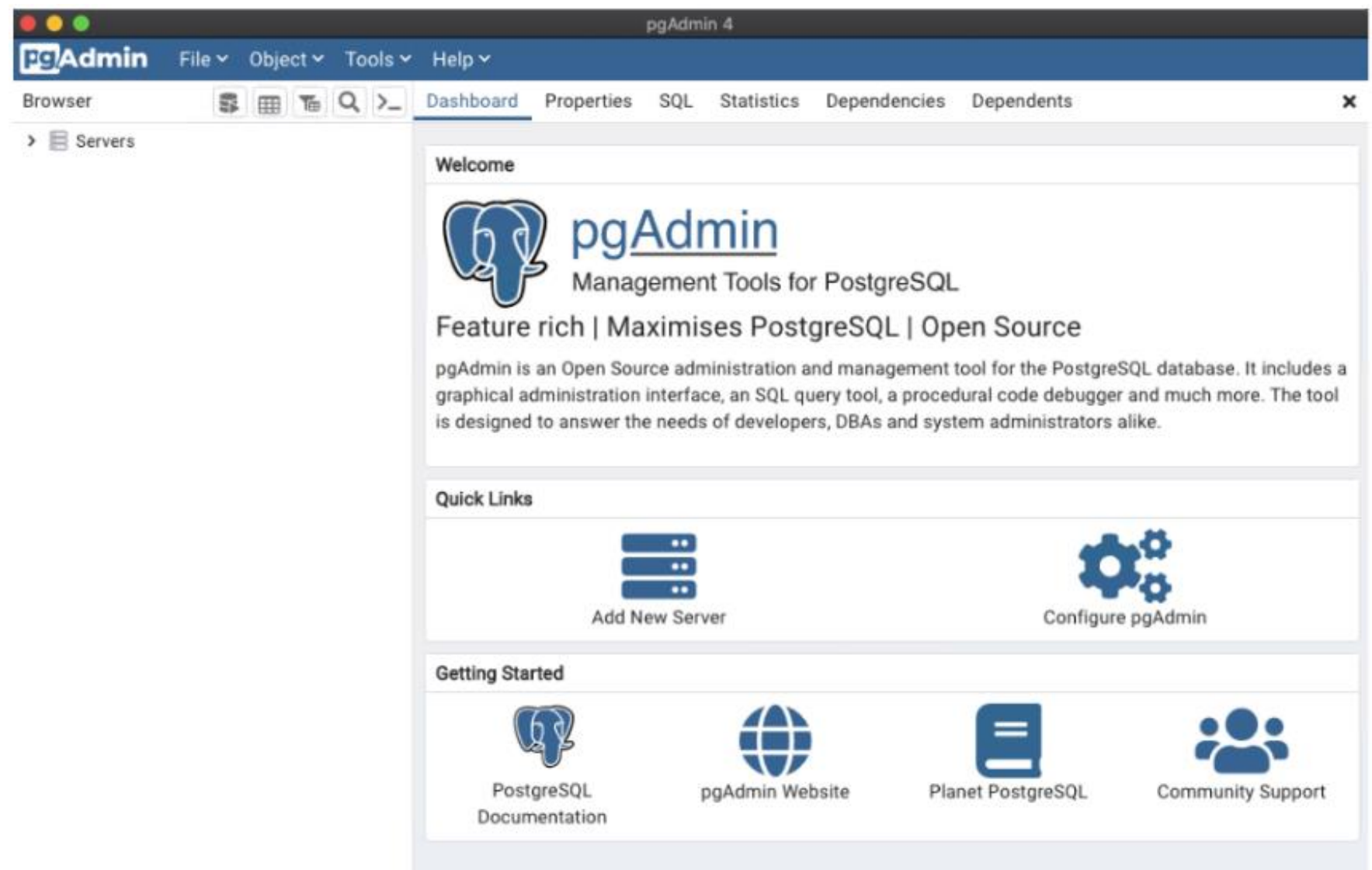
```
insert into t1 values (10,'Hong');  
insert into t1 values (20,'Tom'), (30,'Jane'), (40,'Alice');  
select * from t1;
```

```
update t1 set tname='James' where tno=20;
```

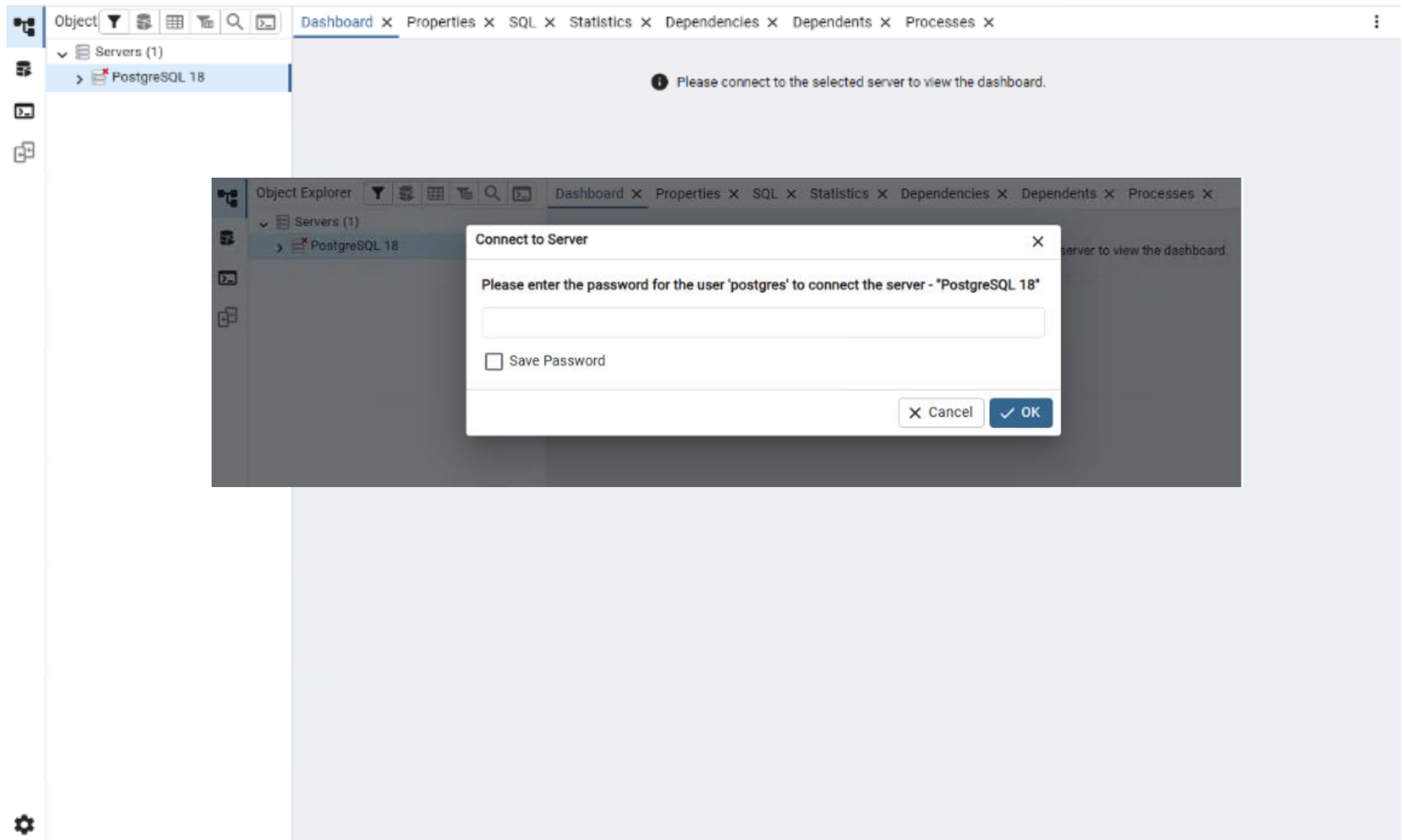
```
delete from t1 where tno = 20;
```

```
\q
```

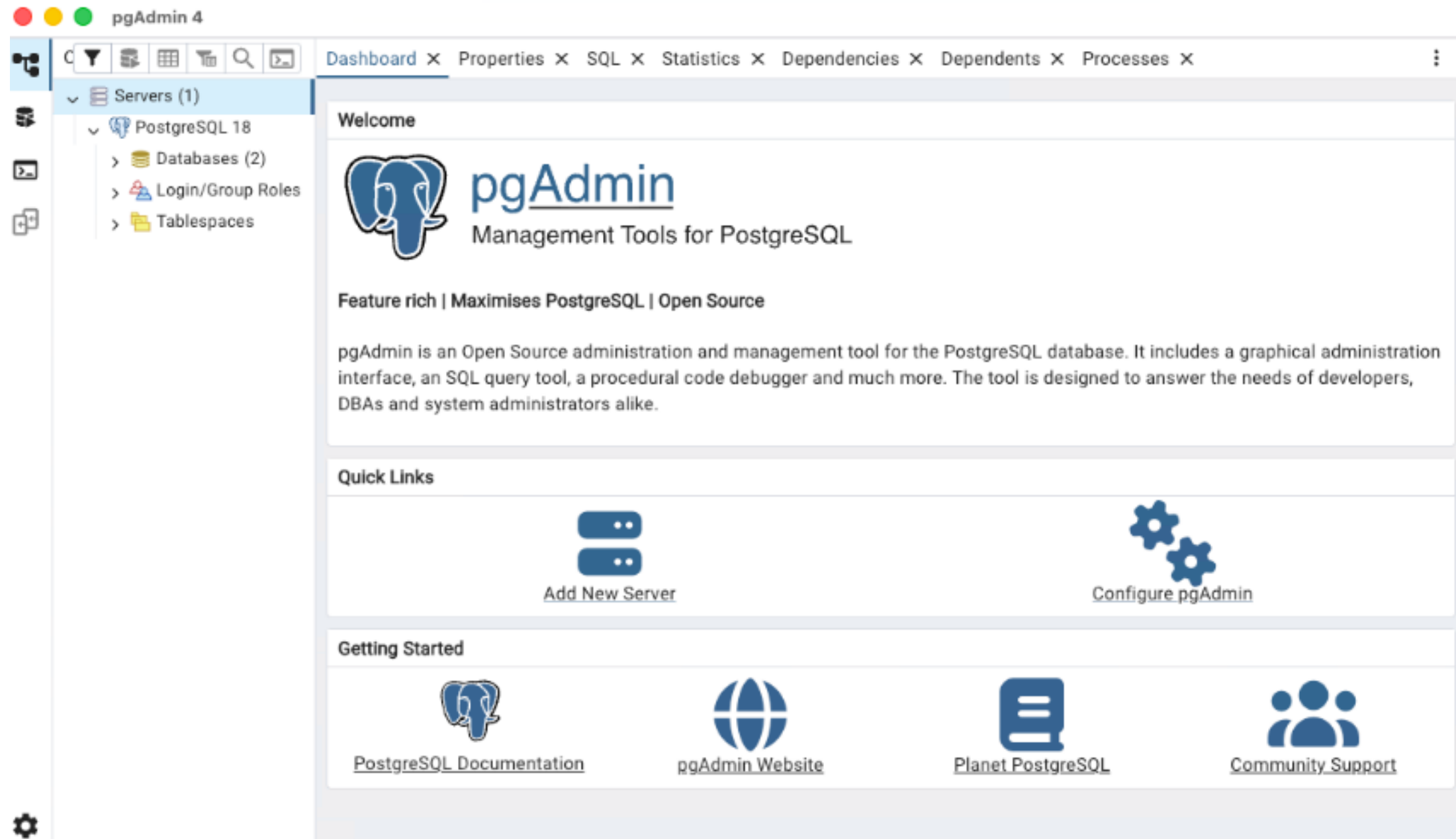
PostgreSQL Client: pgAdmin4



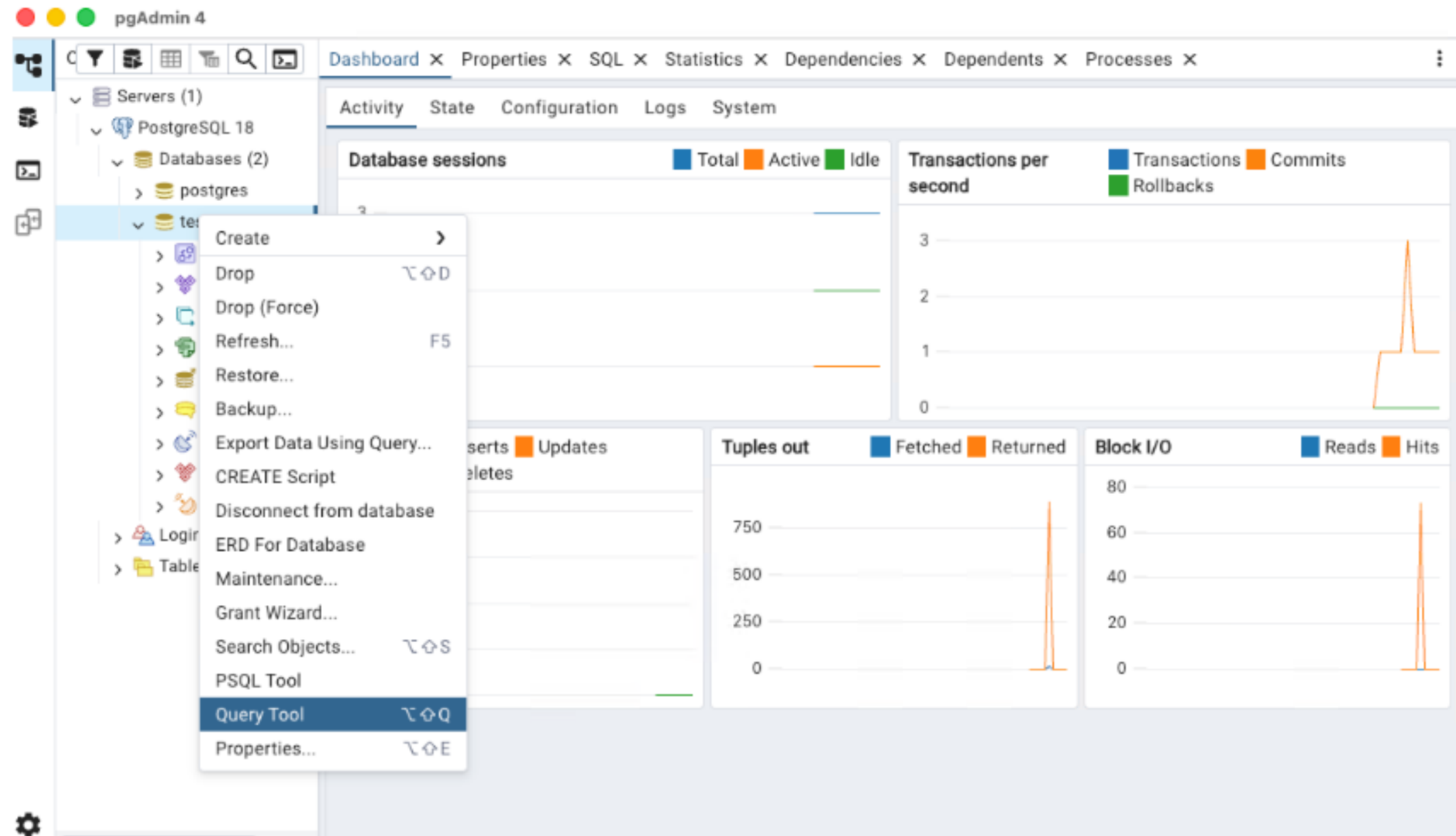
PostgreSQL Client: pgAdmin4



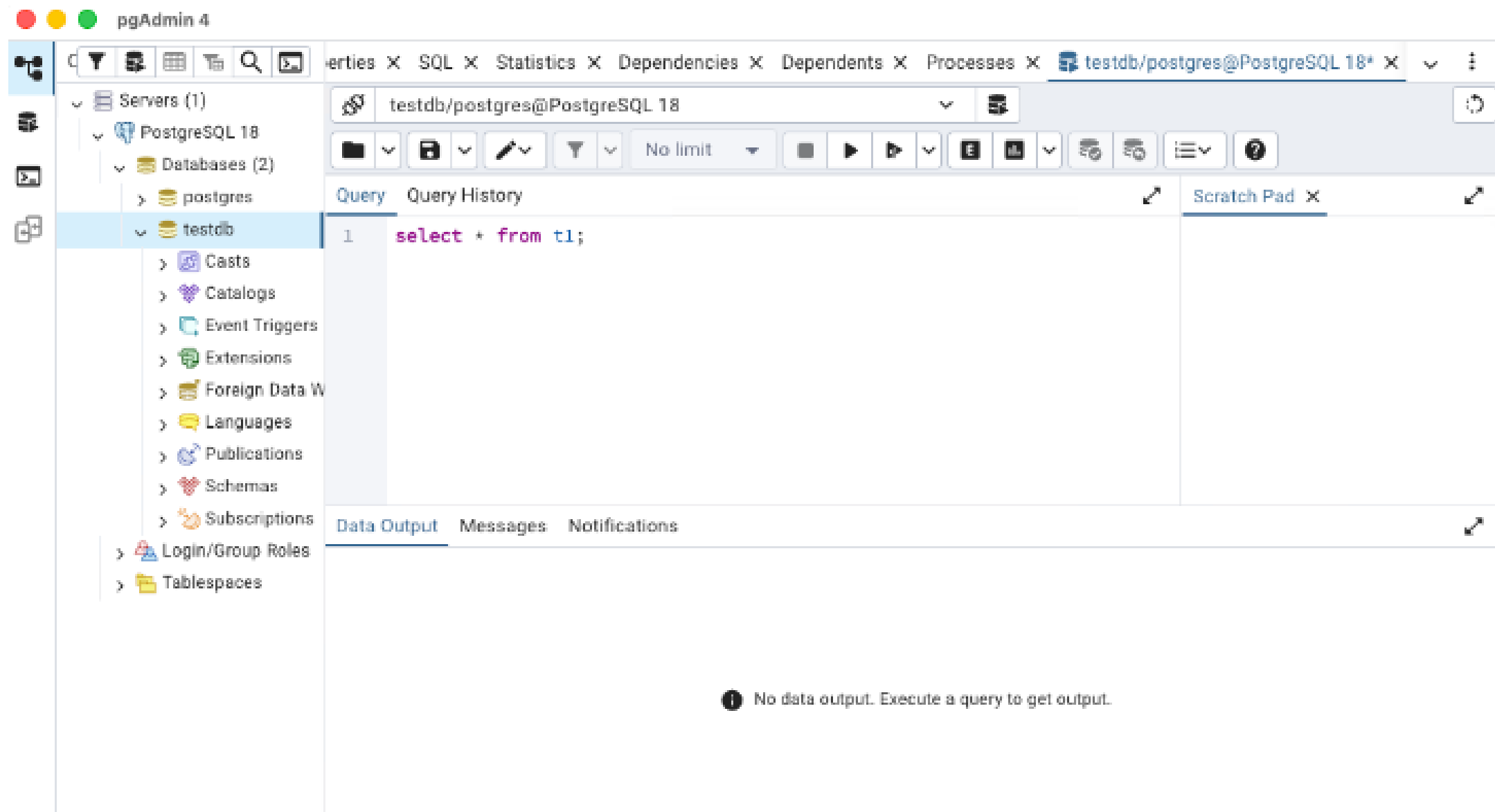
PostgreSQL Client: pgAdmin4



PostgreSQL Client: pgAdmin4

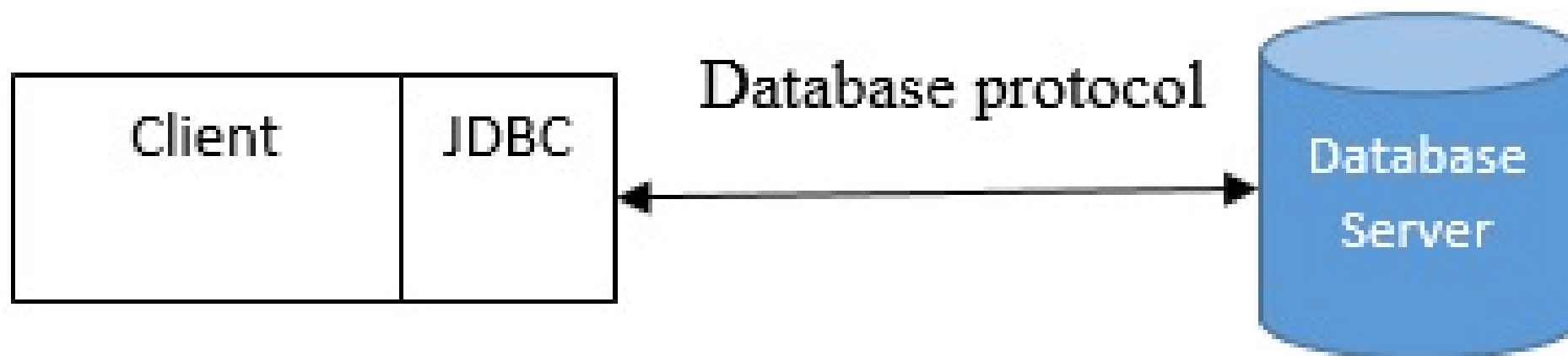


PostgreSQL Client: pgAdmin4



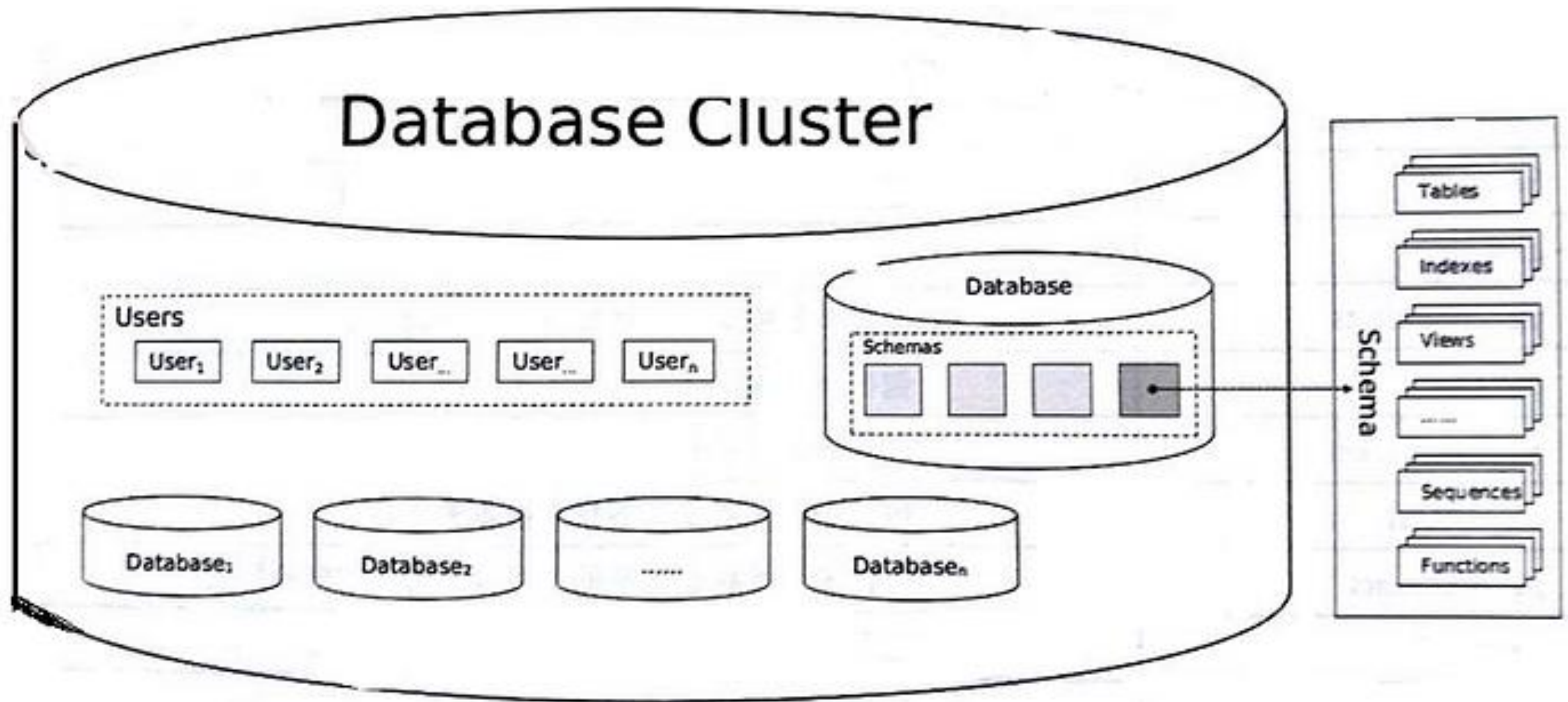
PostgreSQL Client/Server Process

- Process 구조는 Client-Server 모델을 기반으로 함.
- 클라이언트 측에 풍부한 사용자 인터페이스(GUI)를, 서버 측에 데이터베이스를.
- 클라이언트 측에 JDBC 드라이버가 배포.



PostgreSQL 데이터베이스 계층구조

- PostgreSQL 논리적 구조 (Logical Hierarchy)



- 사용자가 데이터를 분류하고 관리하는 관점의 계층적 구조

PostgreSQL 데이터베이스 계층구조

- **데이터베이스 클러스터 (Database Cluster)**

- 하나의 PostgreSQL 서버 인스턴스가 관리하는 데이터 저장소의 최상위 단위.
- 하나의 물리적 디렉터리(PGDATA)에 대응.

- **데이터베이스 (Database)**

- 클러스터 내부에 존재하는 개별 데이터 저장소.
- 스키마, 사용자, 테이블스페이스의 집합.
- 각 데이터베이스는 완전히 독립되어 있어 서로 다른 데이터베이스의 테이블을 직접 조인할 수 없음.

- **스키마 (Schema)**

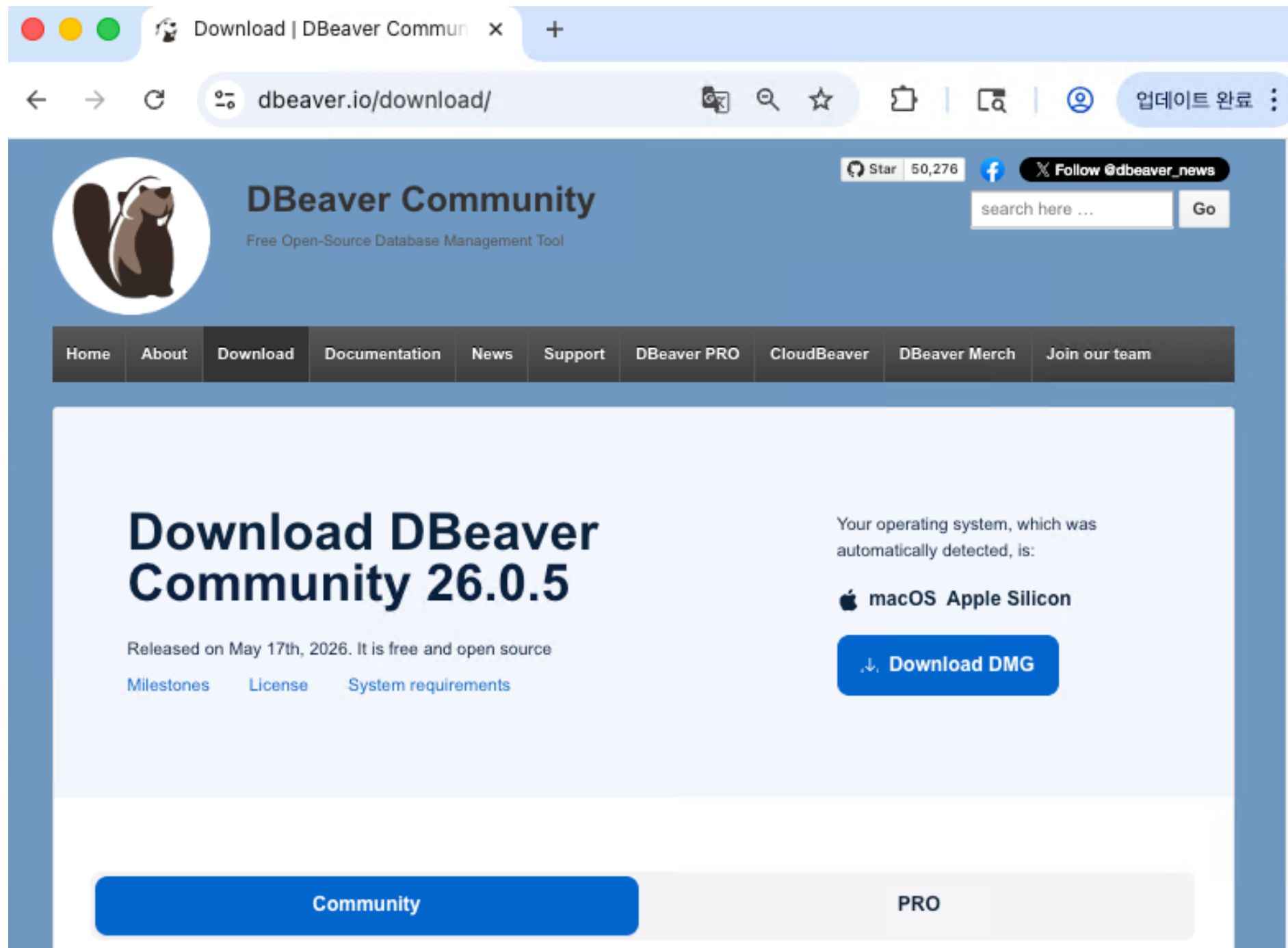
- 데이터베이스 내부의 논리적인 네임스페이스.
- 테이블, 뷰, 인덱스, 함수 등 데이터베이스 객체들을 그룹화하는 논리적 컨테이너 (기본값: public)

- **객체 / 오브젝트 (Objects):**

- 스키마 내에 실제로 존재하는 데이터 및 구조.
- 실제 데이터가 담기는 Table 등의 최하위 단위.
- **테이블 (Table):** 행(Row/Tuple)과 열(Column)로 이루어진 기본 데이터 저장 단위.
- **인덱스 (Index):** 데이터 검색 속도를 높이기 위한 객체.
- **뷰 (View):** 하나 이상의 테이블에서 데이터를 가져오는 가상 테이블.
- **함수/프로시저 (Function/Procedure):** 특정 작업을 실행하는 코드 블록.

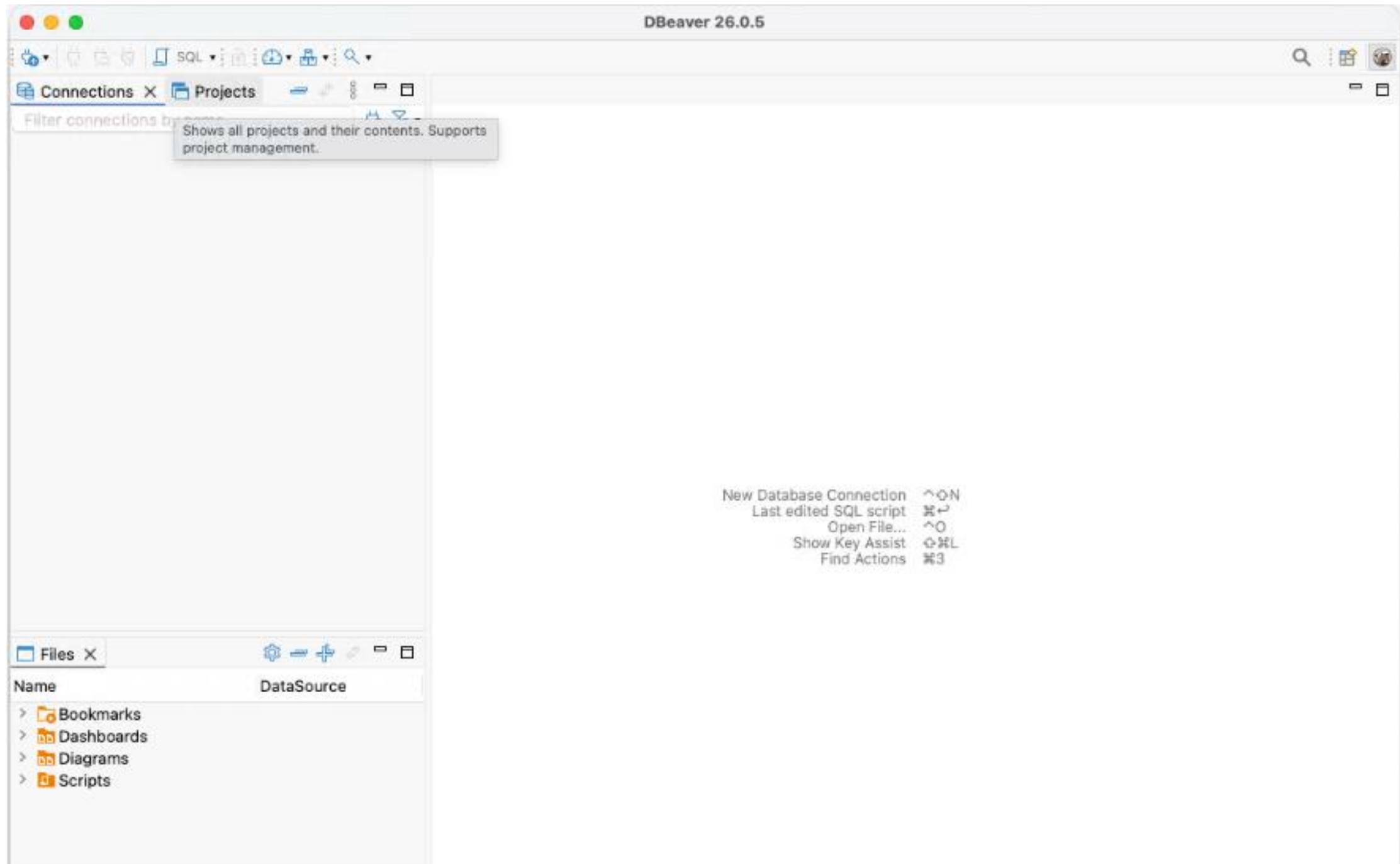
DBeaver

- 다운로드 --> <https://dbeaver.io/download/>

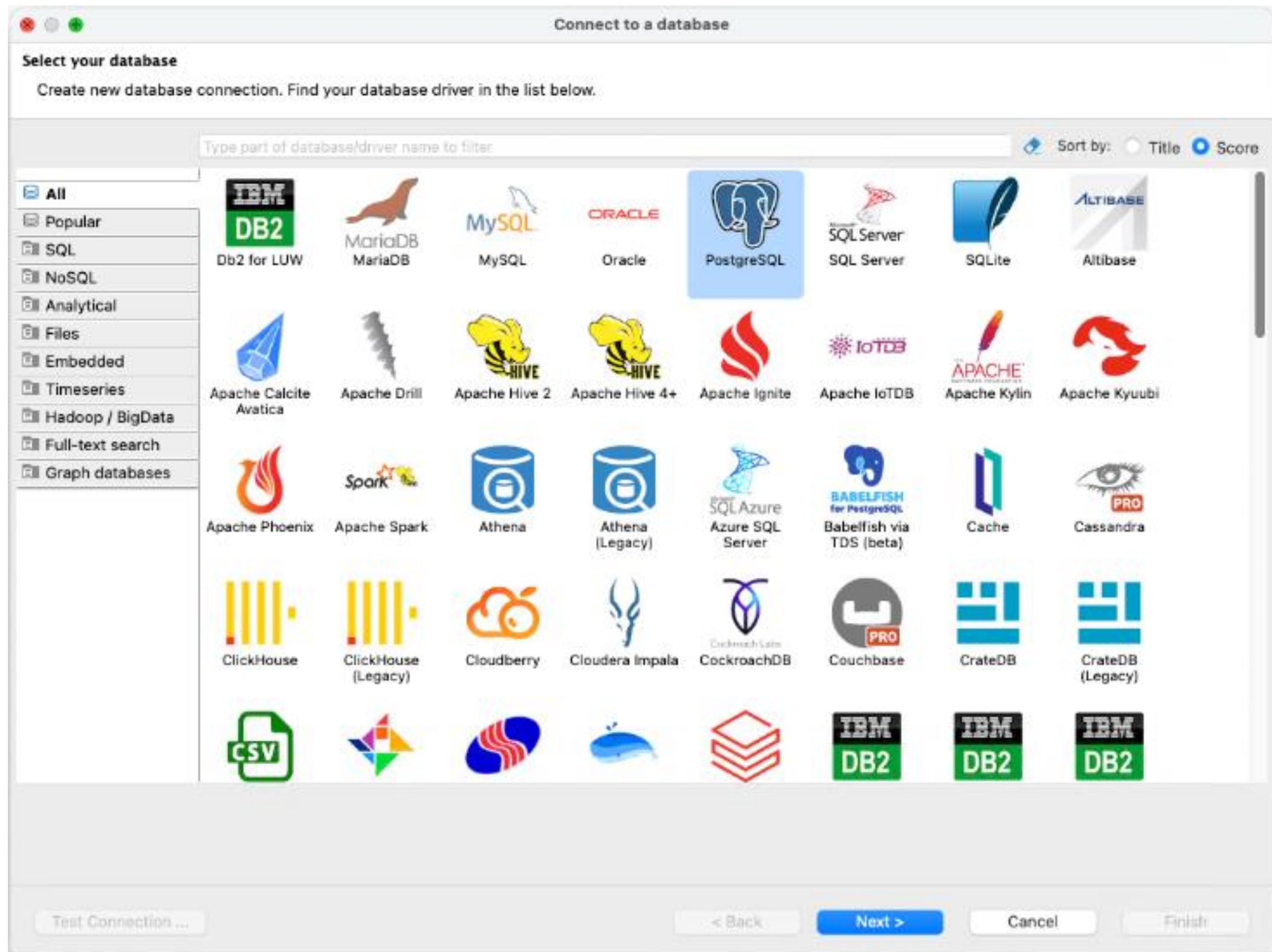


* 신규/샘플 Database 생성 : 아니오

DBeaver



DBeaver



DBeaver

Connect to a database

Connection Settings
PostgreSQL connection settings

Main | Advanced | Driver properties

+ SSH, SSL, ... No profile

Server

Connect by: ☒ Host ☐ URL

URL: jdbc:postgresql://localhost:5432/postgres

Host: localhost Port: 5432

Database: postgres ☒ Show all databases

Authentication

Authentication: Username/password

Username: postgres

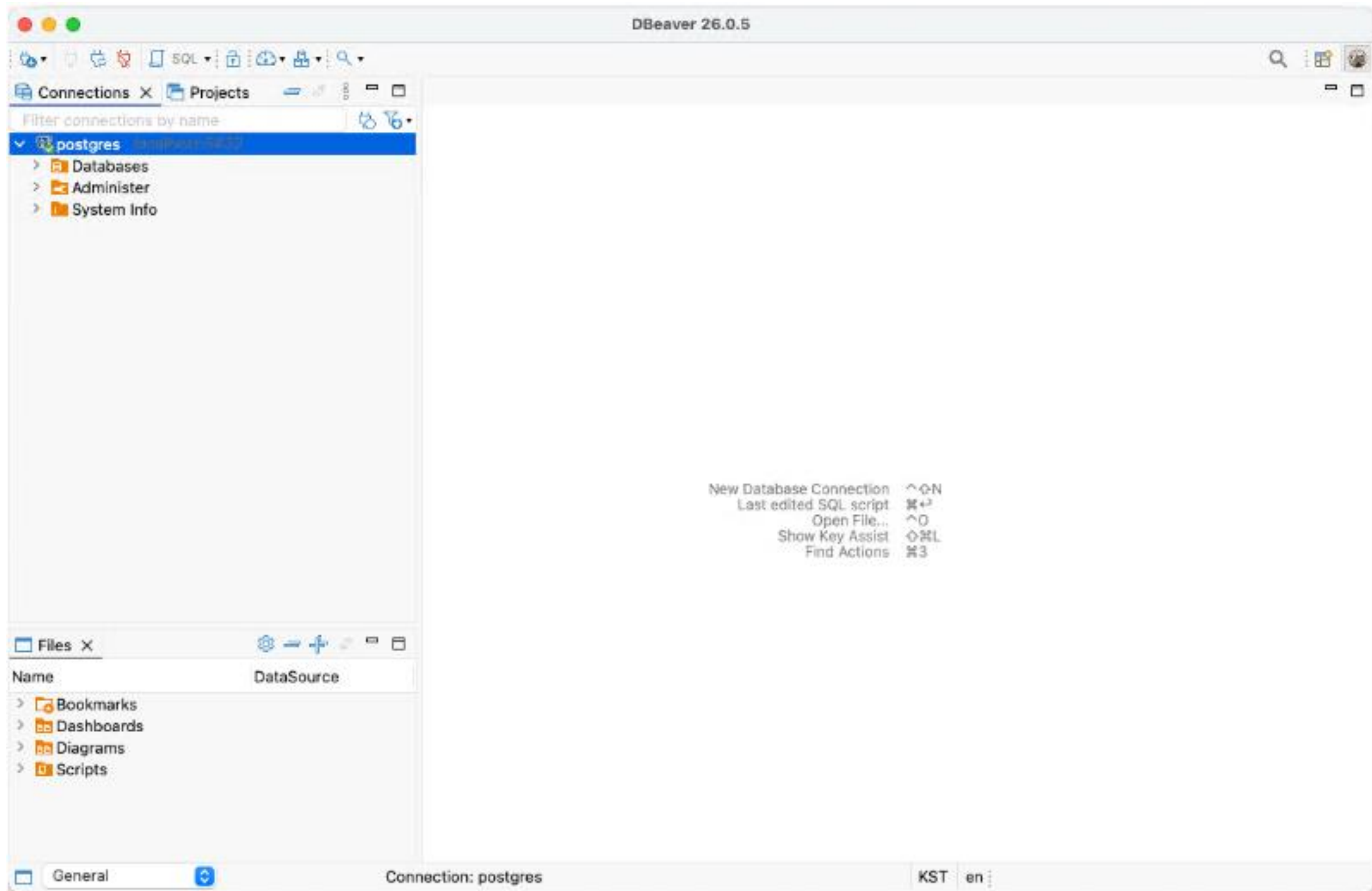
Password: ☒ Save password

[Connection variables information](#) [PostgreSQL](#) Connection details (name, type, ...)

Driver name: PostgreSQL Driver Settings Driver license

Test Connection ... < Back Next > Cancel Finish

DBeaver



PostgreSQL의 주요 데이터형

- 주요 데이터형은 크게 숫자, 문자, 날짜/시간, 불리언, JSON 및 특수 타입으로 분류.
- 객체 관계형 데이터베이스로서 매우 다양하고 강력한 데이터 타입을 제공.
- **숫자 타입 (Numeric Types).**
 - SMALLINT: 2바이트 정수형태로 -32,768에서 +32,767 범위 저장.
 - INTEGER (또는 INT): 일반적인 선택으로 사용되는 4바이트 정수형.
 - BIGINT: 대용량 데이터를 위한 8바이트 정수형.
 - NUMERIC (또는 DECIMAL): 정확한 소수점 연산이 필요한 금융 데이터에 사용되는 가변 소수점 타입.
 - REAL: 4바이트 부동 소수점 실수형.
 - DOUBLE PRECISION: 8바이트 부동 소수점 실수형.
 - SERIAL: 1부터 시작하여 데이터가 삽입될 때마다 자동 증가하는 4바이트 정수형 필드.
- **문자 타입 (Character Types).**
 - VARCHAR(n): 지정한 최대 길이 n 이내의 가변 길이 문자열을 저장.
 - CHAR(n): 지정한 고정 길이 n 을 가집니다. 공백으로 채워져 저장.
 - TEXT: 길이 제한이 없는 가변 길이 문자열을 저장하는 PostgreSQL만의 편리한 타입.

PostgreSQL의 주요 데이터형

- 날짜 및 시간 타입 (Date/Time Types)

- DATE: 연, 월, 일 날짜 정보만 저장.
- TIME: 시, 분, 초 시간 정보만 저장.
- TIMESTAMP: 날짜와 시간을 모두 저장.
- TIMESTAMPTZ: 타임존(Timezone) 정보가 포함된 날짜와 시간. 글로벌 서비스에서 권장.
- INTERVAL: 시간의 간격(예: '1 day', '2 hours')을 저장

- 불리언 및 JSON 타입 (Boolean & JSON Types).

- BOOLEAN: TRUE, FALSE, 또는 NULL 상태를 저장.
- JSON: 텍스트 형태로 JSON 데이터를 그대로 저장. 입력 속도가 빠름.
- JSONB: JSON 데이터를 이진(Binary) 형태로 변환하여 저장. 공백이 제거되고 인덱싱을 지원하여 검색 및 연산 속도가 훨씬 빠름.

- 특수 및 고급 타입 (Special Types)

- UUID: 글로벌 고유 식별자(UUID)를 저장하는 데이터 타입.
- BYTEA: 이미지, 파일 등의 이진 파일(Binary Data)을 저장.
- ARRAY: 동일한 데이터 타입의 값을 배열 형태로 묶어 저장 가능.
- INET / CIDR: IPv4 또는 IPv6 형태의 네트워크 주소를 효율적으로 저장하고 검증.

PostgreSQL의 시간관련 내장 함수

함수	설명
NOW()	현재 날짜 + 시간
CURRENT_TIMESTAMP	현재 날짜 + 시간
CURRENT_DATE	현재 날짜
CURRENT_TIME	현재 시간
LOCALTIME	시간만(타임존 제외)
LOCALTIMESTAMP	날짜+시간(타임존 제외)

Select 문 구조

```
SELECT [DISTINCT] { * | column명 | 표현식 }  
FROM 테이블명;
```

- 테이블 전체 검색

```
SELECT *  
FROM emp;
```

- 특정 컬럼 검색

```
SELECT empno , ename  
FROM emp;
```


연산 표현식

- 연결 연산

- 컬럼 또는 문자를 다른 컬럼과 연결할 때 사용
- 두 개의 세로 선 (||)으로 표시

- 산술 연산

- 종류 : 더하기 (+), 빼기 (-), 곱하기(*), 나누기 (/)
- 산술연산자를 사용하여 숫자나 날짜 데이터에 대한 계산
- 괄호를 사용해서 연산의 우선순위 조정가능

Column 별칭

- Column의 에 대한 별도의 이름
- Column명 다음에 AS를 쓴 뒤 그 뒤에 별칭으로 사용할 단어를 제시 (AS는 생략 가능)
- 특수 문자나 공백을 칼럼의 별칭으로 사용하고자 하면 별칭에 큰 따옴표(“ ”)를 사용

중복 행 제거

- Distinct를 사용하여 같은 Column에 있는 동일한 값은 한번만 출력
- SELECT 바로 뒤에 사용
- Distinct 다음에 여러 열을 지정 가능
 - 선택한 모든 열에 영향을 주어 결과로 고유한 열 조합이 표시

Data 정렬

- 정순(오름차순) 또는 역순(내림차순)으로 정렬(기본값 :ASC)
- ORDER BY 절은 항상 SELECT문의 마지막에 작성

Null

- 사용할 수 없고, 지정되지 않고, 적용할 수 없는 값
- 0 또는 space이 아니며, 공간의 개념이 없음
- Null 값과의 연산 결과는 무조건 Null

Data 필터링(where)

SELECT [DISTINCT] { * | column명 | 표현식 }

FROM 테이블명

[WHERE condition(s)]

Limit ~

Offset ~;

- WHERE절을 통해 Data 검색 제한
- 문자열과 날짜 값은 작은 따옴표(")로 묶어서 표시
- 모든 행 검색 시, WHERE절 생략

조건의 유형(Type)

- Point 조건
 - =
- 범위 조건
 - >, <, >=, <=, between ~ and ~
- 열거 조건
 - in (,,)
- 상태 조건
 - is null, is not null
- 패턴 조건
 - like _, like %

논리 연산자

- And : 양쪽 조건이 모두 True이면, True
- Or : 양쪽 조건 중 하나만 True 이면, True
- Not : 뒤의 조건이 False이면 True
- Where 절에서 조건의 수가 여러 개이면
 - And, Or 를 통해 연결할 수 있다.

함수

종류	설명
문자 함수	문자열 처리
숫자 함수	숫자 계산
날짜 함수	날짜/시간 처리
변환 함수	데이터 타입 변환
NULL 처리 함수	NULL 처리
조건 함수	조건 분기
집계 함수	여러 행 집계
윈도우 함수	분석 함수
JSON 함수	JSON 처리
배열 함수	ARRAY 처리
시스템 함수	DB 정보 조회
수학 함수	수학 계산
정규식 함수	REGEXP 처리

Group by

```
SELECT  그룹함수( [ DISTINCT ] /ALL),  
        {COLUMN ,...}  
FROM    테이블명  
[WHERE  조건                ]  
[GROUP BY 컬럼1, 컬럼2 .... ]  
[HAVING 그룹 조건          ]  
[ORDER BY 컬럼1, 컬럼2 ....]
```


Join

- 여러 테이블의 데이터를 질의 할 경우 Join 을 사용

```
SELECT table1.column , table2.column,...
```

```
FROM      table1 , table2
```

```
WHERE      table1.column1 = table2.column2;
```

- Join 표기 방식

- 일반 Join
- ANSI 표준 Join

- Join의 종류

- Equi Join, Non-Equi Join, Self Join
- Inner Join, Outer Join

ANSI Join

SELECT *table1.column, table2.column*

FROM *table1*

[**CROSS JOIN** *table2*] |

[**NATURAL JOIN** *table2*] |

[**JOIN** *table2* **USING** (*column_name*)] |

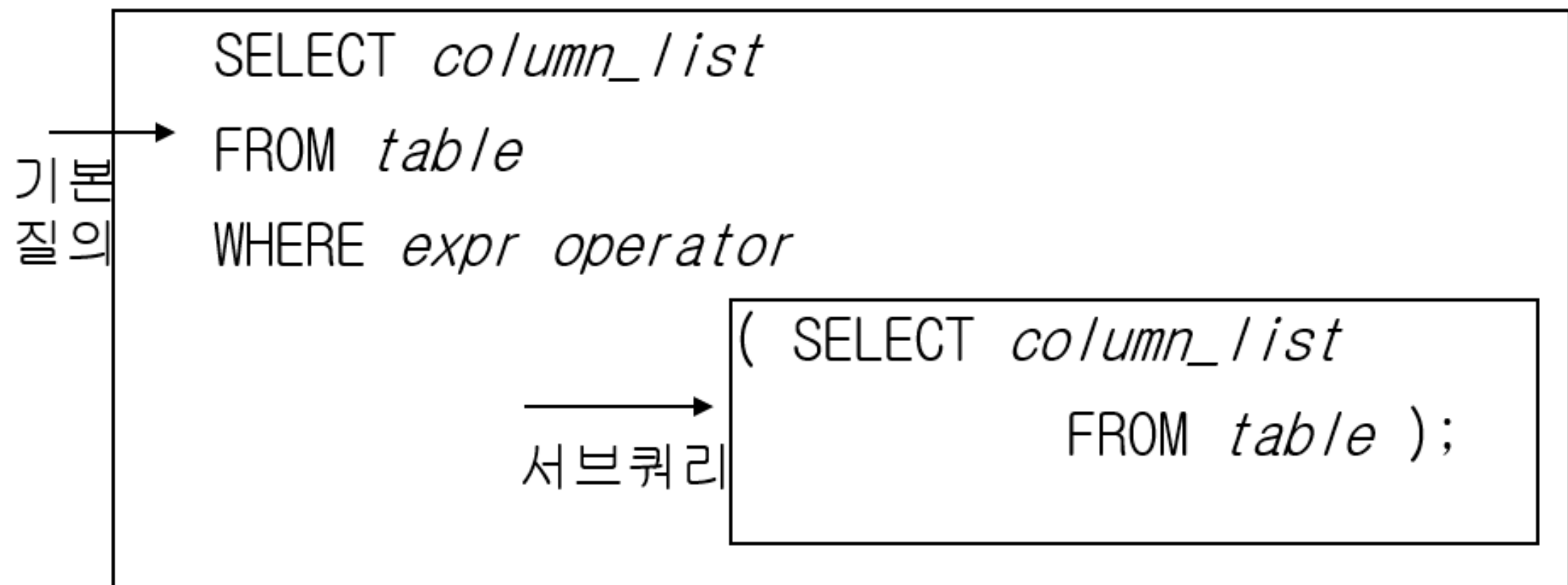
[**JOIN** *table2* **ON**(*table1.column_name=table2.column_name*)] |

[**LEFT** | **RIGHT** | **FULL** [**OUTER**] **JOIN** *table2*

ON(*table1.column_name=table2.column_name*)];

Sub Query

- 여러 메인 쿼리 안에서 또 다른 쿼리문이 있는 것
- 주로 SELECT 문에서 많이 사용.
- INSERT, UPDATE, DELETE 문에서도 사용가능.
- 종류: 단일행, 다중행, 다중열



CTE(Common Table Expression)

- 임시적으로 사용할 수 있는 쿼리 블록을 생성
- WITH 키워드를 사용하여 정의
- 다중 CTE 사용 가능 (쉼표로 구분)

```
WITH 임시_테이블명 AS (  
  
    SELECT 컬럼1, 컬럼2  
    FROM 실제_테이블명 WHERE 조건  
  
)
```

```
SELECT * FROM 임시_테이블명;
```

DML

- Insert (입력)

```
INSERT INTO table [( column [, column...] )]  
VALUES (value [, value...] );
```

- Update (수정)

```
UPDATE table  
SET column = value [, column = value, ... ]  
[WHERE condition]
```

Returning;

- Delete (삭제)

```
DELETE FROM table  
[WHERE condition];
```

집합연산자(Set Operators)

- 각 SELECT 문은 컬럼 수와 데이터 타입이 일치해야 함

연산자	설명	특징
UNION	두 집합을 합치되 중복 제거	정렬 발생, 중복 제거
UNION ALL	두 집합을 단순히 합침 (중복 허용)	빠름, 중복 데이터 유지
INTERSECT	두 집합의 교집합 반환	정렬 발생, 중복 제거
EXCEPT	첫 번째 집합에서 두 번째 집합을 뺀	정렬 발생, 중복 제거

Table 구성과 제약조건

- **CREATE TABLE** DDL(데이터 정의 언어) 문을 사용
- 스키마, 테이블 이름, 컬럼(데이터 타입 및 제약조건)으로 구성.

```
CREATE TABLE [스키마명.]테이블명 (  
    컬럼명1 데이터타입 [제약조건],  
    컬럼명2 데이터타입  
    [제약조건], ... );
```

- 예)

```
CREATE TABLE users  
(  
    id SERIAL PRIMARY KEY,  
    username VARCHAR(50) NOT NULL UNIQUE,  
    email VARCHAR(100),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

- SERIAL: 자동 증가하는 정수형 타입 (주로 기본키에 사용)
- PRIMARY KEY: 고유 식별자 (중복 불가, NULL 불가)
- NOT NULL: 필수 입력값 지정
- UNIQUE: 고유한 값만 허용 (중복 불가)
- DEFAULT: 기본값 설정

데이터 무결성을 위한 제약 조건

구분	제약조건
기본키 제약	테이블내의 각 row를 식별하기 위한것으로 null 이나 중복된값을 허용하지 않는다.
외래키 제약	해당 속성은 참조되는 테이블의 속성 값들 중에 하나와 일치하거나 null만을 가질수 있다.
Unique 제약	테이블의 모든 row에 대해서 유일한 값을 가져야 한다.
Not Null 제약	Not Null로 설정된 속성은 Null을 가질 수 없다.
Check 제약	해당 속성에 저장 데이터의 값의 타입이나 범위가 지정된 조건을 벗어날 수 없다.

Table 구조 변경

- 테이블 구조를 변경하려면 주로 ALTER TABLE 명령어를 사용.
- 컬럼 추가 (ADD)
 - ALTER TABLE 테이블명 ADD 컬럼명 데이터타입;
- 컬럼 삭제 (DROP)
 - ALTER TABLE 테이블명 DROP COLUMN 컬럼명;
- 컬럼 이름 변경 (RENAME)
 - ALTER TABLE 테이블명 RENAME COLUMN 기존컬럼명 TO 새컬럼명;
- 컬럼 데이터 타입 변경 (ALTER / TYPE)
 - ALTER TABLE 테이블명 ALTER COLUMN 컬럼명 TYPE 새로운데이터타입;

View

- 물리적으로 데이터를 저장하지 않고, 하나 이상의 테이블에서 데이터를 가져와 가상으로 보여주는 **논리적인 테이블**

```
CREATE OR REPLACE VIEW 뷰_이름  
AS  
SELECT 수정된_컬럼1, 수정된_컬럼2  
FROM 테이블_이름  
WHERE 조건_문;
```

- **WITH CHECK OPTION (값의 범위 제한)**

PL/pgSQL (Procedural Language/PostgreSQL)

- PostgreSQL에서 제공하는 **절차형(Procedural) SQL 언어**
 - 일반 SQL만으로는 어려운 다음 기능들을 사용할 수 있음.
 - 변수 선언
 - IF 문
 - CASE 문
 - LOOP 문
 - FOR 문
 - WHILE 문
 - 예외 처리(EXCEPTION)
 - 함수 호출
 - 프로시저 작성

Function

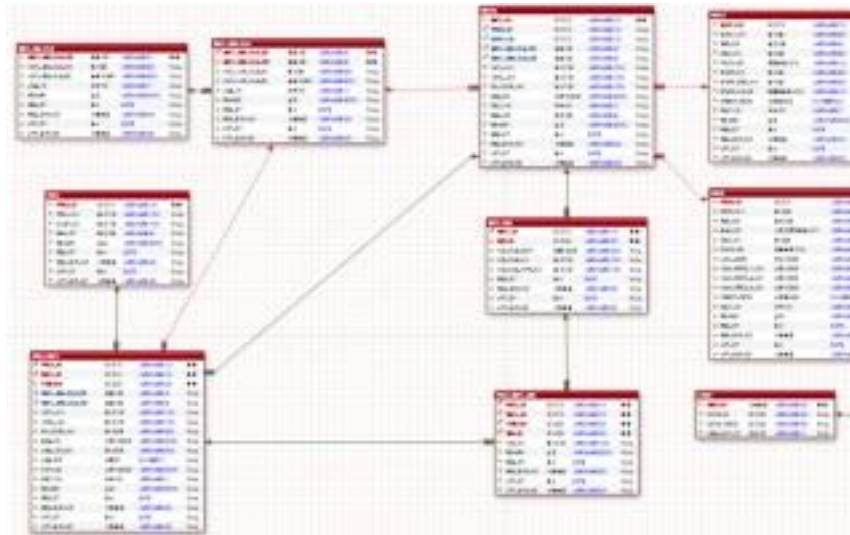
- 데이터베이스 서버에 저장되고 실행되는 SQL 및 절차적 언어(PL/pgSQL 등)의 코드 블록

```
CREATE [OR REPLACE] FUNCTION 함수이름(매개변수명 데이터타입, ...)
    RETURNS 반환타입
LANGUAGE 언어이름
AS $$
DECLARE
    -- [선택 사항] 변수 선언
    변수이름 데이터타입 := 초기값;
BEGIN
    -- 실행할 SQL 및 로직 작성
    RETURN 반환값;
END;
$$;
```

Stored Procedure

- 데이터베이스 서버에 미리 저장해 두고 필요할 때 호출하여 실행할 수 있는 SQL 및 절차적 코드(변수 선언, 제어문 등)의 집합

```
CREATE [OR REPLACE] PROCEDURE 프로시저_이름(파라미터_목록)
LANGUAGE plpgsql
AS $$
DECLARE
    -- 변수 및 상수 선언 (생략 가능)
BEGIN
    -- 실행할 SQL 및 로직 작성
    -- COMMIT 또는 ROLLBACK 사용 가능
END;
$$;
```



DATABASE의 설계

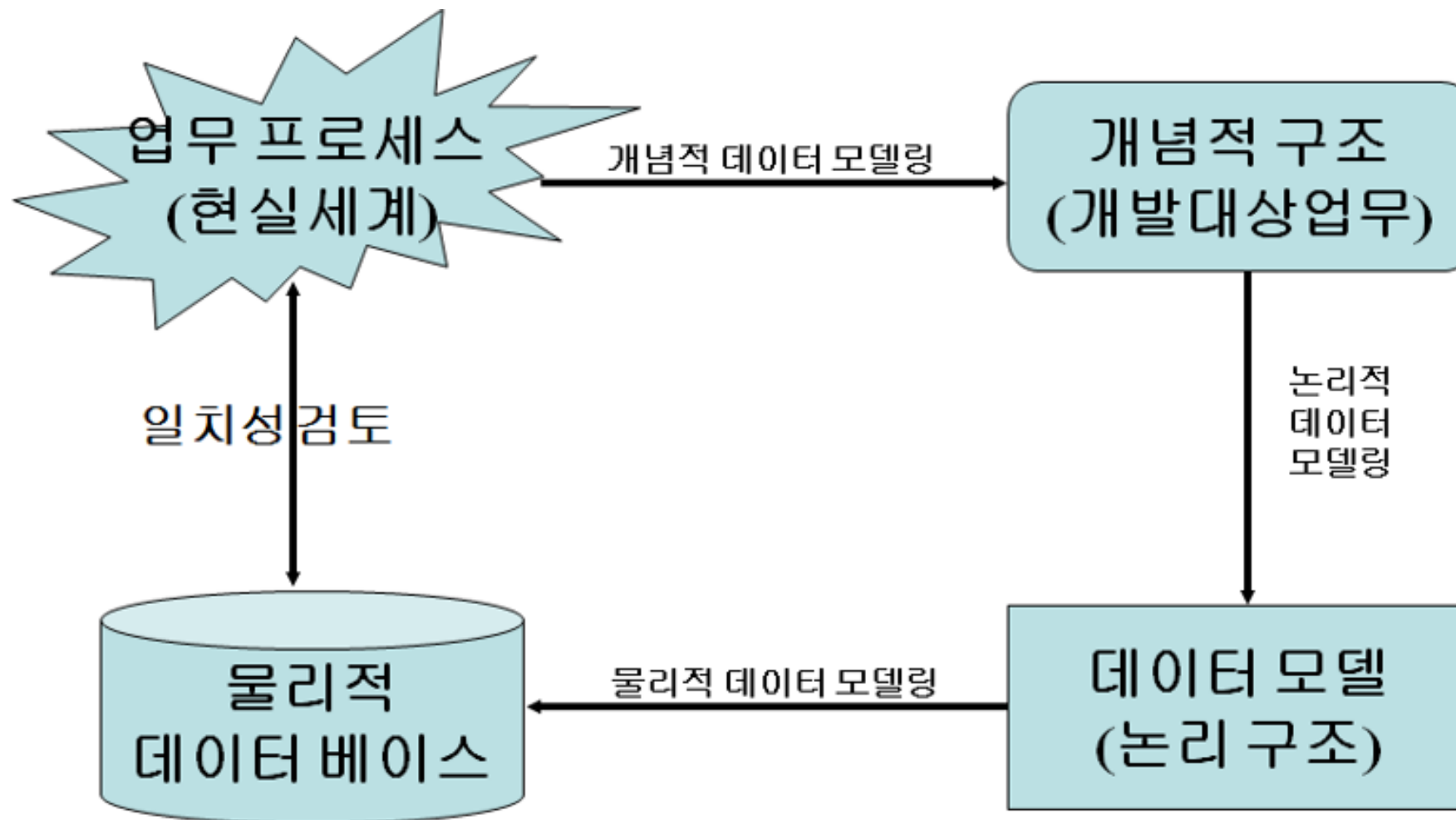
Relation Ship(관계)

고객번호	이름	주민번호	연락처	메일주소	...	...
1000	홍길동	700503-1000000	010-0000-0000	h@daum.net		
1001	Alan Turing	030913-3000000	011-1111-1111	at@daum.net		
1002	김연아	721007-2000000	010-2222-2222	k@naver.com		
1003	Irving John Good	730504-1000000	016-3333-3333	j@outlook.com		
1004	박지성	700528-1000000	010-4312-8471	p@naver.com		
1005	Florence Nightingale	981211-2000000	010-4589-6439	fn@naver.com		
1006	Ada Lovelace	050802-4000000	010-4727-7850	al@gmail.com		
1007	유재석	780502-1000000	010-7845-4190	s@gmail.com		
1008	이도	100421-3000000	010-1114-3924	d@outlook.com		
1009	이지은	780102-2000000	010-8206-5957	u@outlook.com		

차량번호	차종	차명	색깔	연식	판매일	고객번호
23하4567	SUV	GV90	그린	2026	2026-12-21	1000
108나7890	세단	현대 쏘나타	그린	2021	2021-10-12	1001
32다9876	SUV	싼타페	그레이	2026	2026-02-24	1002
78마5678	세단	에쿠스	화이트	2020	2020-01-24	1004
77아1234	세단	그랜저	실버	2023	2023-06-27	1007
108자5678	해치백	아반떼	블루	2024	2024-02-08	1008

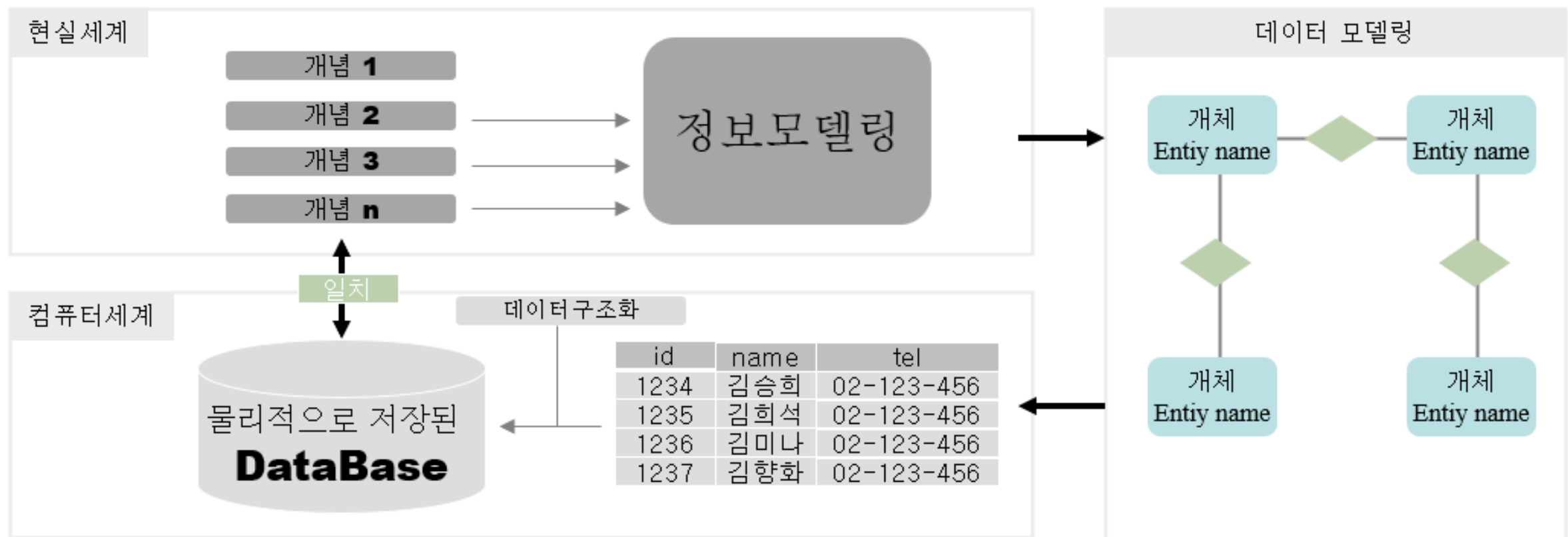
Database 모델링

- 현실 세계의 업무 프로세스와 Data를 추상화하여, System에 Database로 구조화 시키는 과정

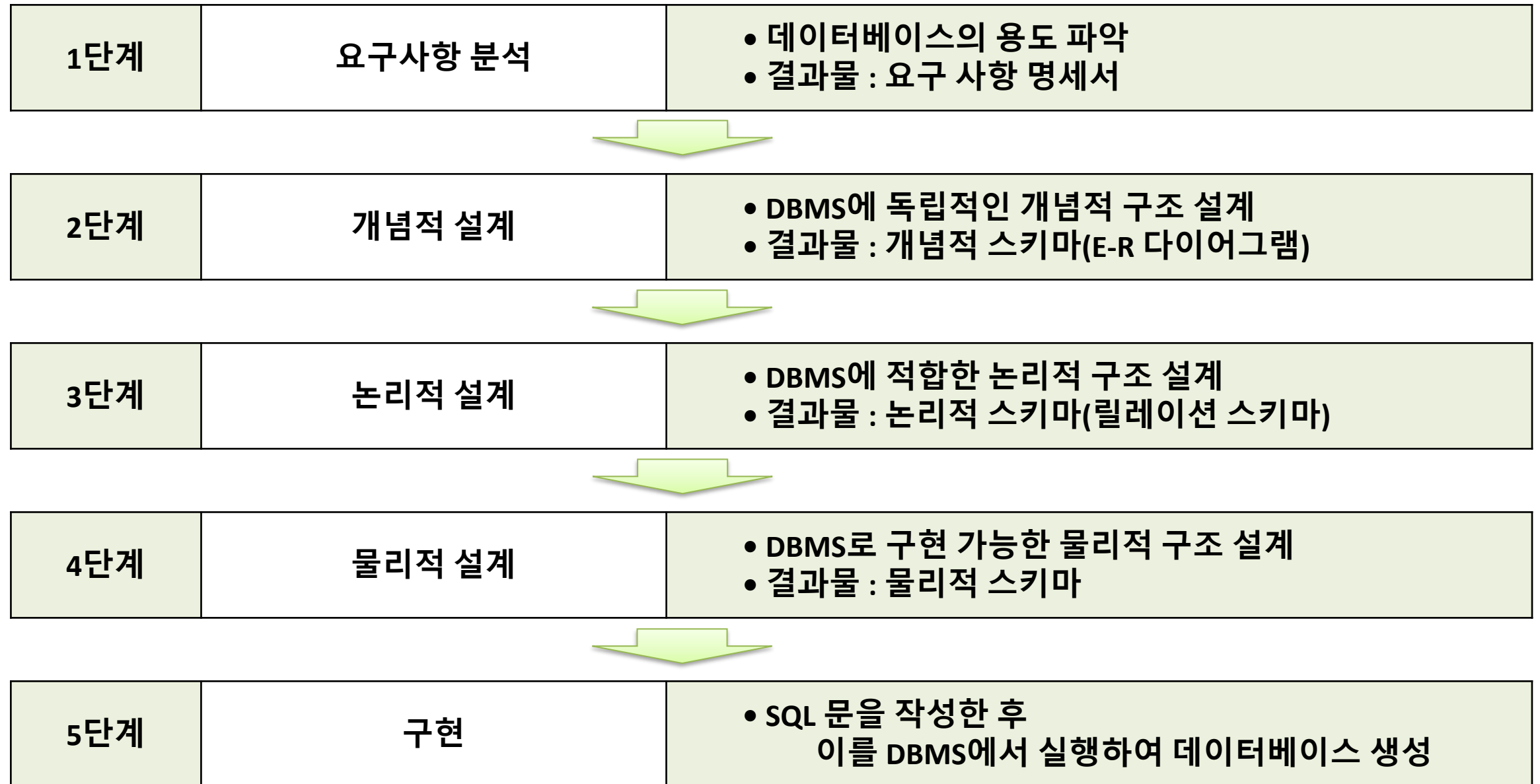


Database 모델링

- 데이터 모델링이라고도 하며, 다른 말로는 개체-관계 모델링.
- 현실의 개념들을 체계적으로 수집하여 정보모델링을 통해 사용자의 정보요구사항을 조사하고, 이를 **개체, 관계, 속성**을 중심으로 명확하게 체계적으로 표현하고 문서화 하는 기법.



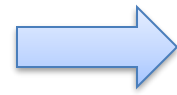
데이터 베이스 설계 순서



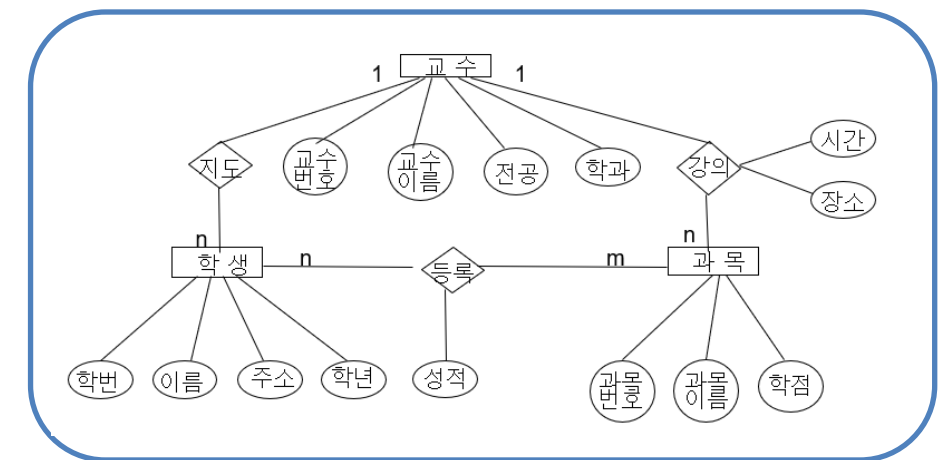
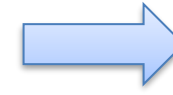
개념적 설계

- 개념적 모델링 (Conceptual Data Modeling)
 - 세계에 존재하는 개체 (Entity)들 중에 데이터베이스에 저장할 대상들을 추출하여 이를 추상화하여 표현하는 과정.

업무 프로세스
(현실 세계)



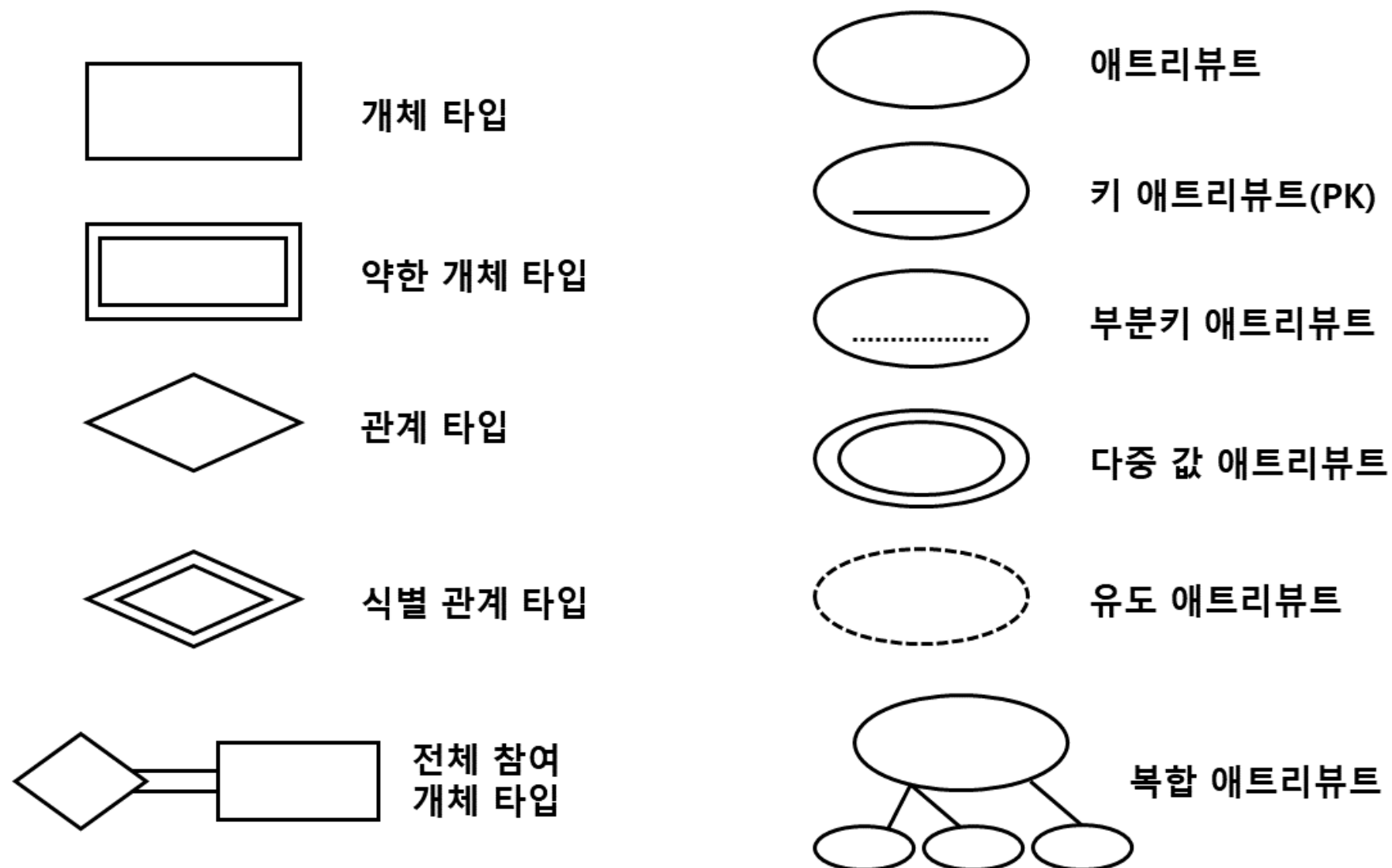
1. 실체 정의
2. 속성 정의
3. 식별자 정의
4. 관계 정의
5. 차수성 정의
6. 선택성 정의



개념적 데이터 모델링

- peter-chen 다이어그램 표기

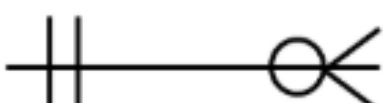
- 세계에 존재하는 개체 (Entity)들 중에 데이터베이스에 저장할 대상들을 추출하여 이를 추상화하여 표현하는 과정.



개념적 데이터 모델링

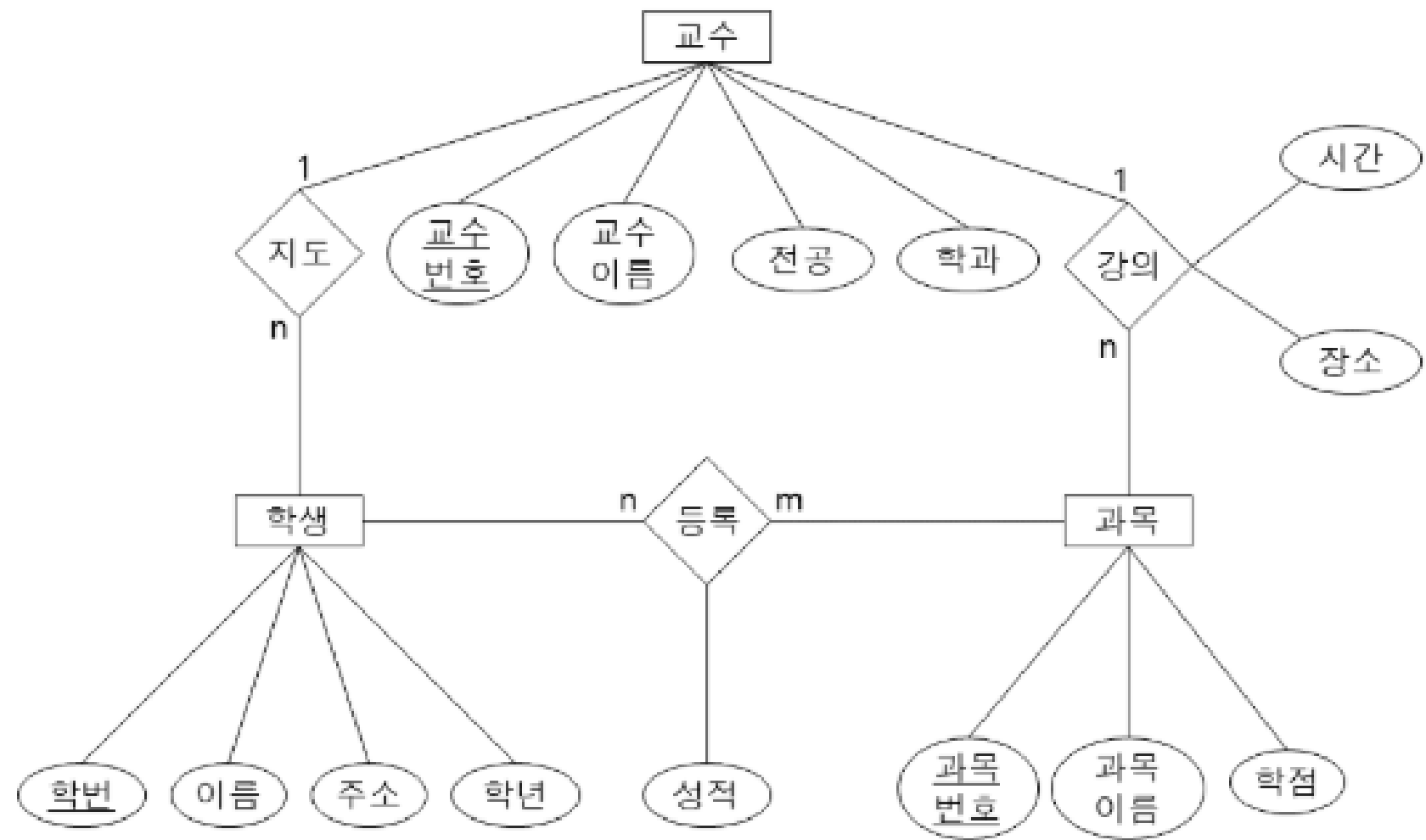
● 관계 유형 표기

- 사상수 표기(Crow's Foot Notation) : 가장 많이 사용.
- Peter Chen 표기 : 1976년에 제안된 원조 표기.

관계유형	사상수(Crow's Foot)	Chen 표기법
필수일 (mandatory one)		
선택일 (optional one)		
필수다 (mandatory many)		
선택다 (optional many)		
1대N 관계 (one-to-many)		

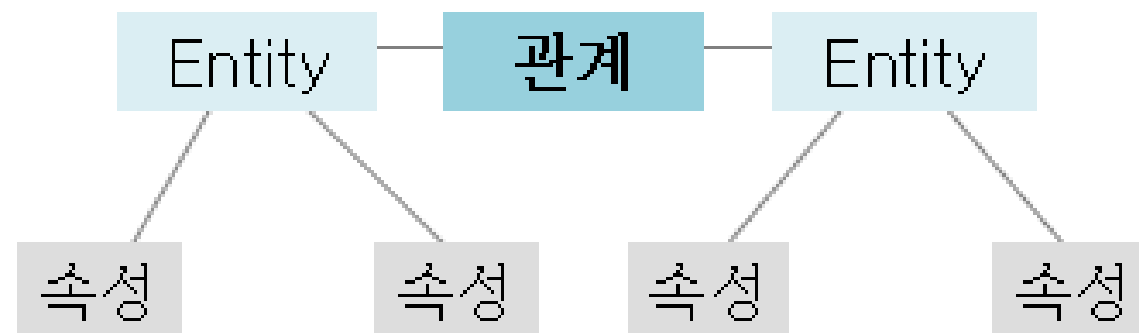
개념적 데이터 모델링

1. 실체 정의
2. 속성 정의
3. 식별자 정의
4. 관계 정의
5. 차수성 정의
6. 선택성 정의



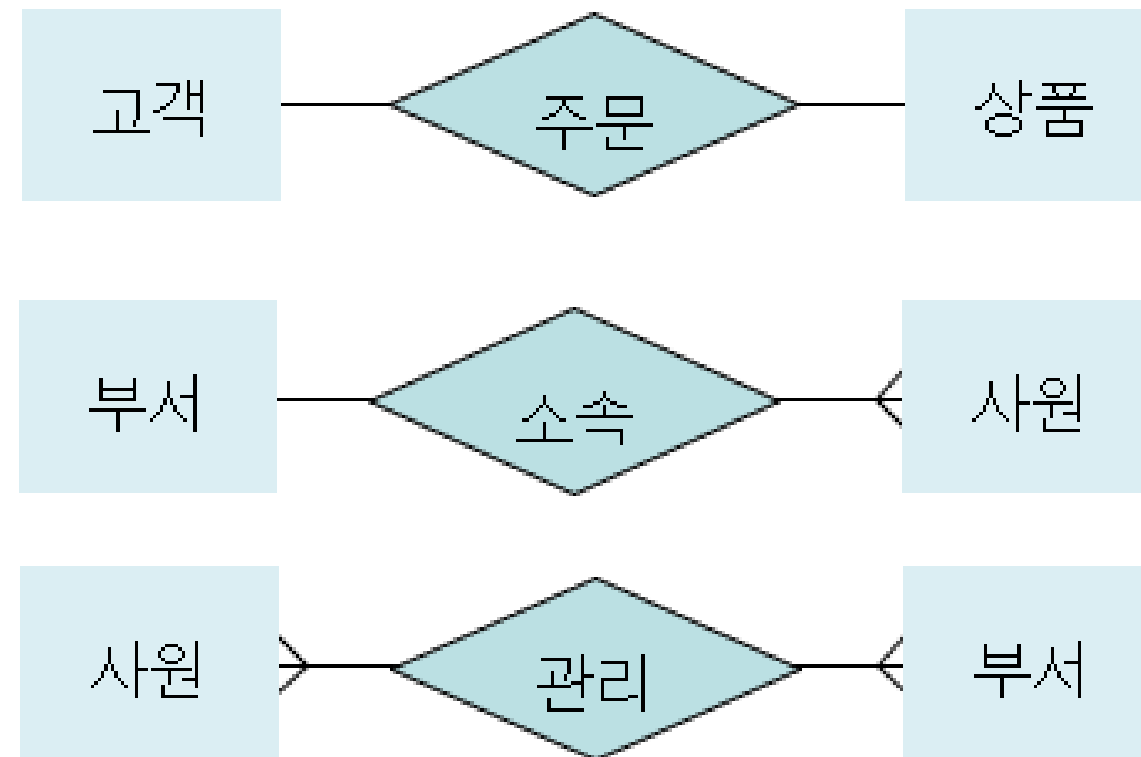
개념적 데이터 모델링

- **실체(Entity) : 강한, 약한**
 - 저장되고 관리되어야 할 정보(명사)로서 물리적으로는 DMBS의 테이블에 해당.
- **속성(Attribute) : 기초, 추출, 설계**
 - Entity를 설명하는데 사용되는 명사로 Column명을 의미.
- **식별자(UID : Unique Identifier) : 후보, 주, 부, 외부, 대리**
 - 중복을 배제함을 의미.
- **관계(Relational)**
 - 두 개의 실체간에 명명되어진 의미 있는 연결이며, 두 실체간에 업무적인 연관성을 나타냄.

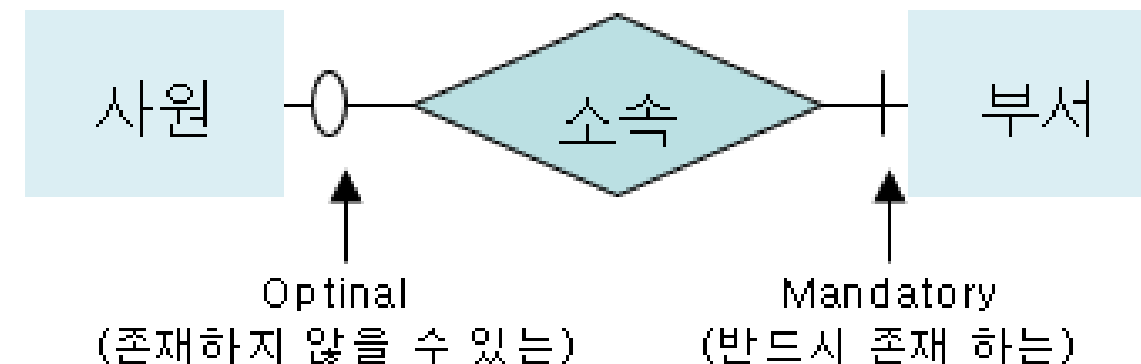


개념적 데이터 모델링

- 차수성 (Cardinality) : [1:1] , [1:N] , [M:N], [Exactly]



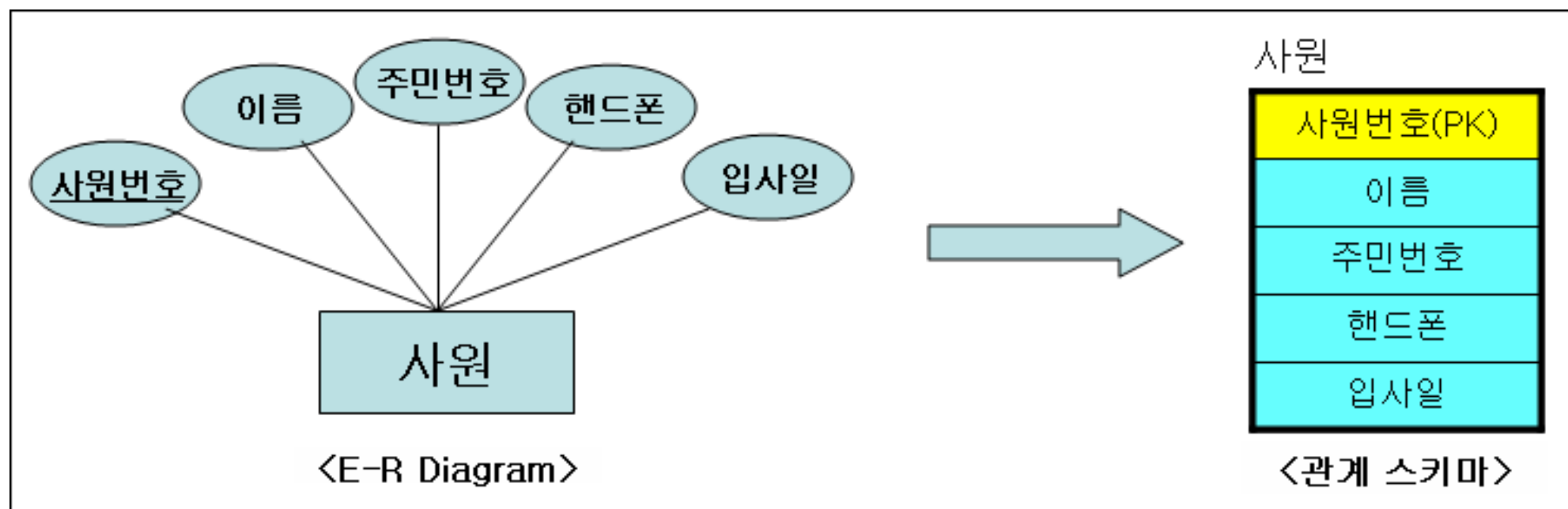
- 선택성 (Optional, Mandatory)



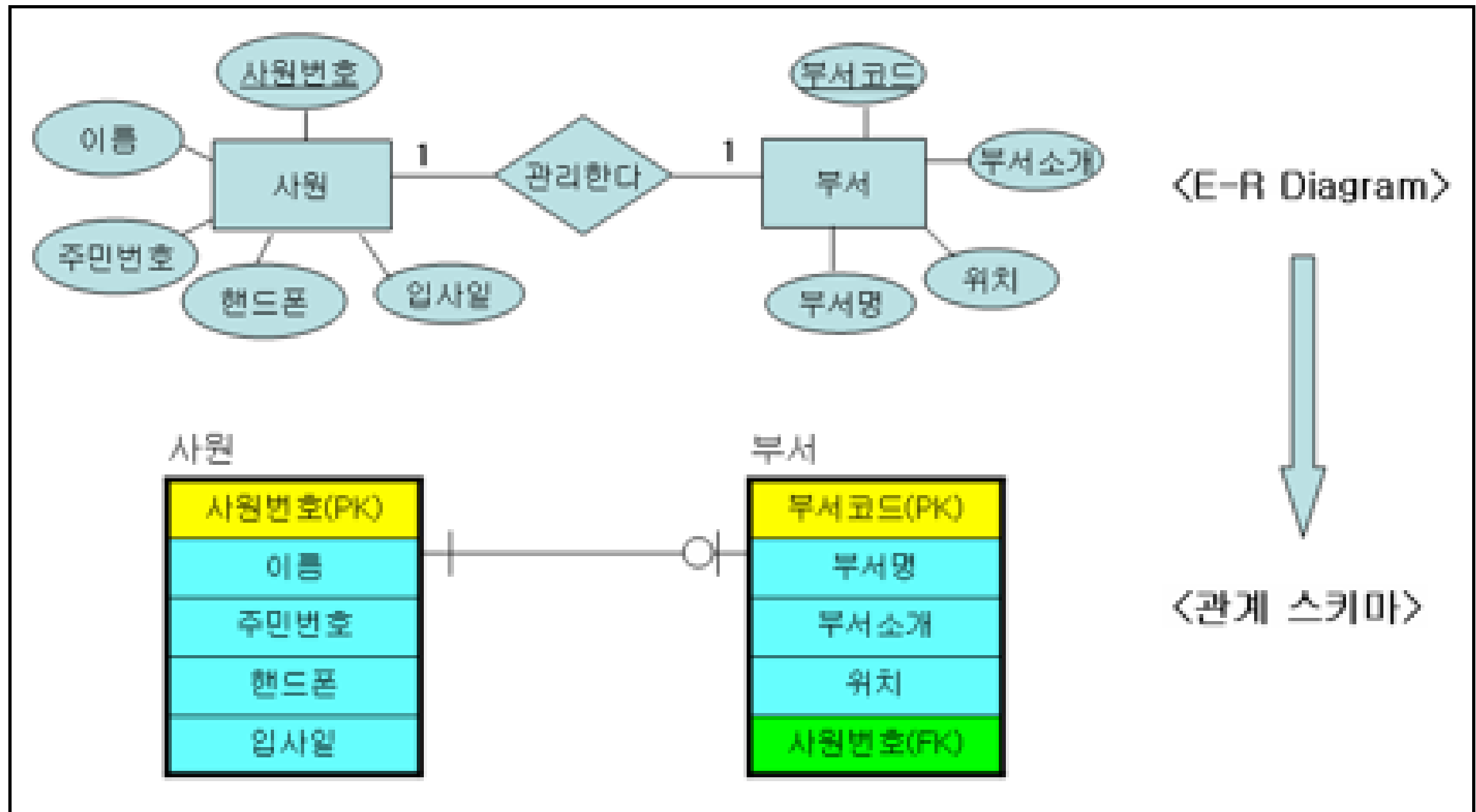
논리적 데이터 모델링

● Mapping Rule 4단계

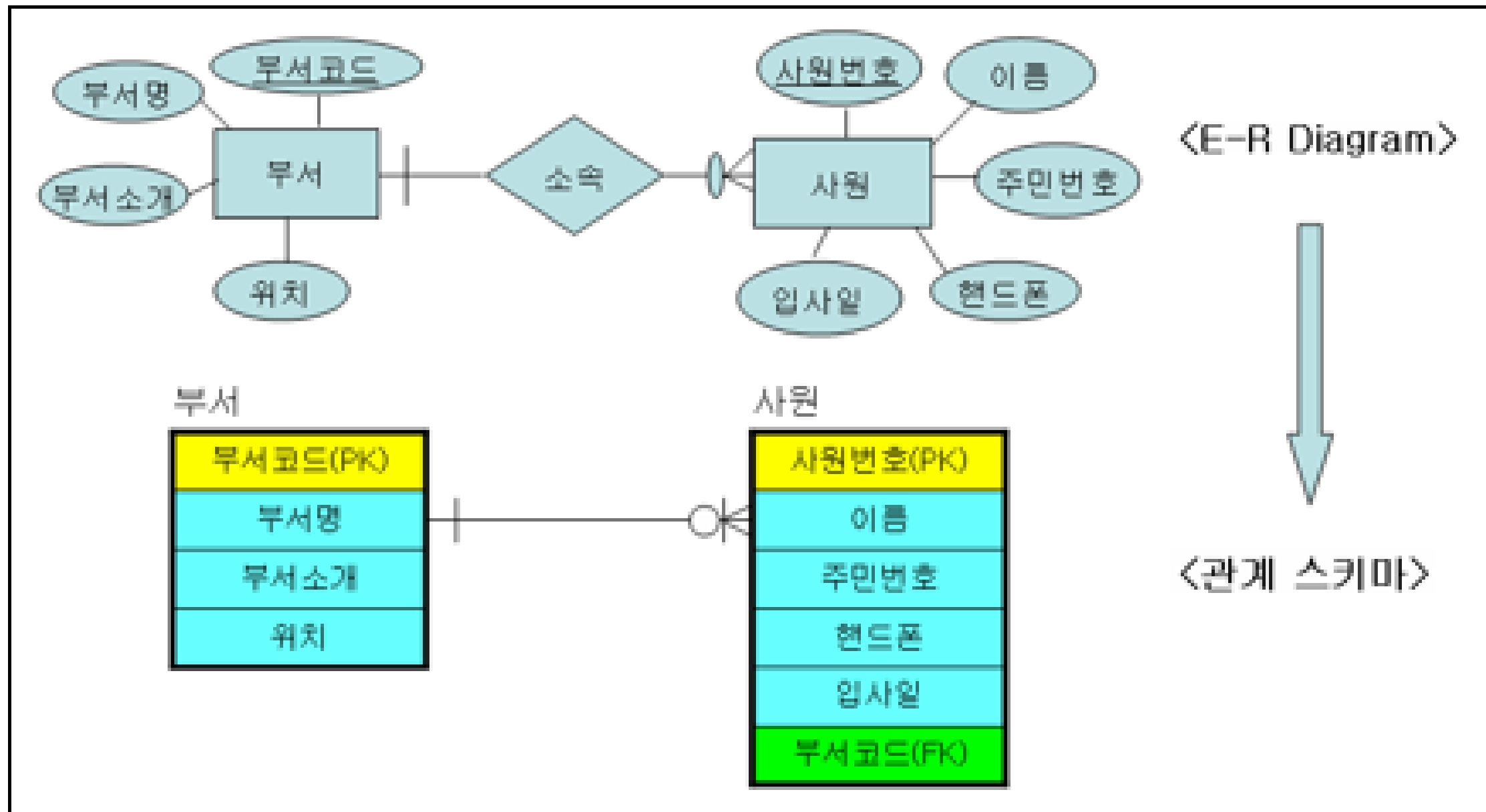
- 1단계 : 실체(Entity) → Table(테이블)
- 2단계 : 속성(Attribute) → Column(컬럼)
- 3단계 : 주식별자(Primary UID) → Primary Key(기본키)
- 4단계 : 관계(Relational) → Foreign Key(외래키)



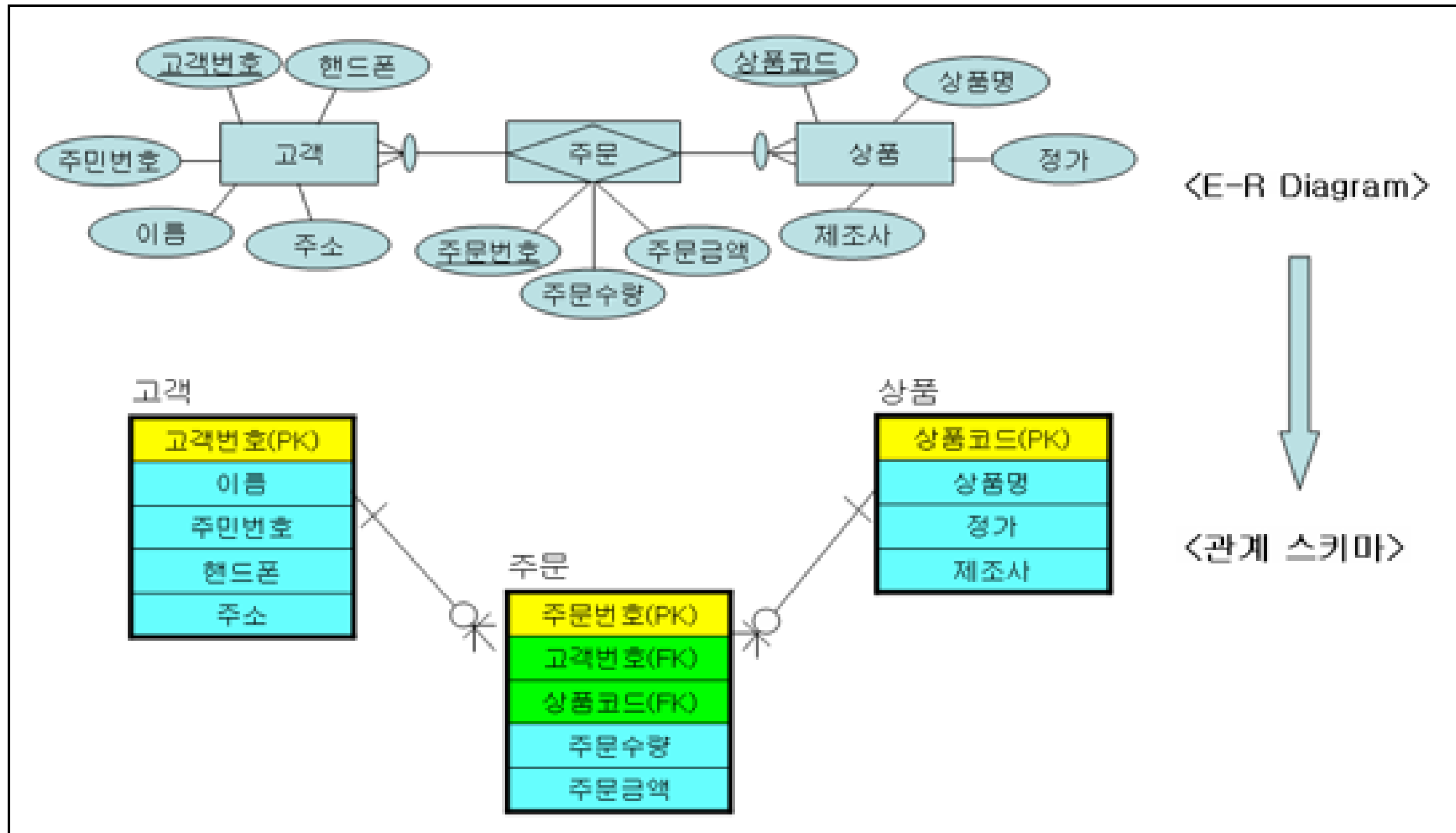
논리적 데이터 모델링



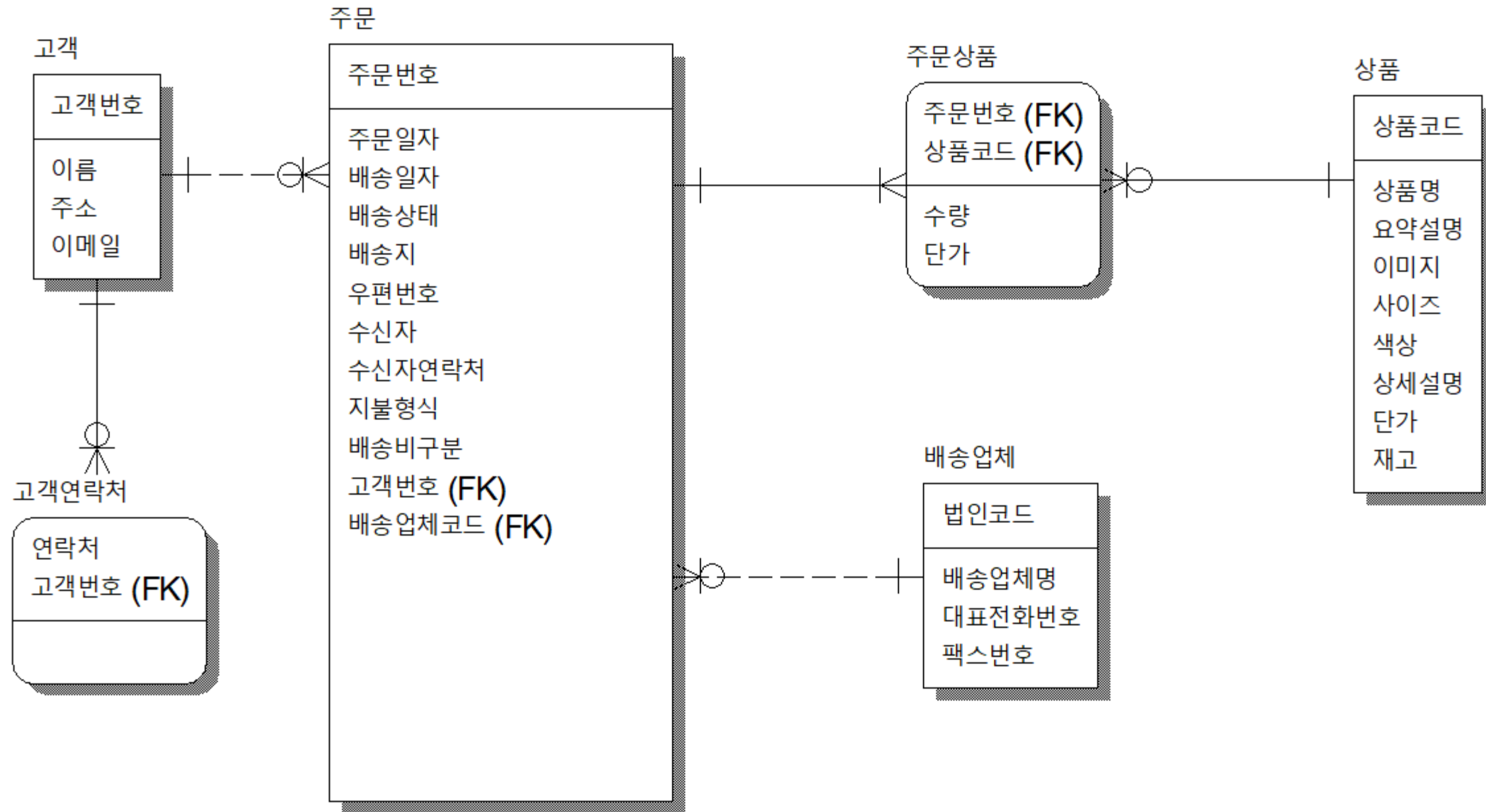
논리적 데이터 모델링



논리적 데이터 모델링



논리적 데이터 모델링



데이터 베이스 정규화

● 정규화

- 정보공학 방법론(Information Engineering)의 핵심 도구
- 1981년 영국의 컴퓨터 과학자 제임스 마틴(James Martin)과 클리프 핀켈스타인(Clive Finkelstein) 창안

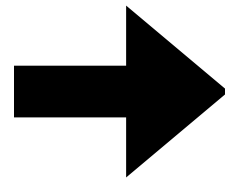
정규화 절차	내용
제1정규화(1NF)	속성(Attribute)의 원자성을 확보하고 원자값이 아닌 도메인을 분해한다.
제2정규화(2NF)	기본키가 2개 이상의 속성으로 이루어진 경우, 부분 함수 종속성을 제거 한다.
제3정규화(3NF)	기본키를 제외한 컬럼간의 종속성을 제거한다. 즉 이행 함수 종속성을 제거한다.
BCNF	기본키를 제외하고 후보키가 있는 경우, 후보키가 기본키를 종속시키면 분해한다.
제4정규화(4NF)	여러 컬럼들이 하나의 컬럼을 종속시키는 경우, 분해하여 다중값 종속성을 제거한다.
제5정규화(5NF)	조인에 의해서 종속성이 발생하는 경우, 분해한다.

관계형 데이터베이스의 설계에서 중복을 최소화하게 데이터를 구조화하는 프로세스

제1 정규화(1 Normal Form)

기본키를 설정하고, 모든 도메인이 더 이상 분해되지 않은
원자값으로 구성

학번	성명	동아리
1	김남준	밴드, 사진
2	김석진	댄스
3	장희수	독서
4	박지민	낚시, 등산

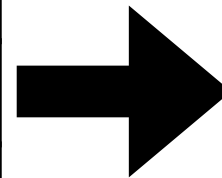


학번	성명	동아리
1	김남준	밴드
2	김석진	댄스
3	장희수	독서
4	박지민	낚시
1	김남준	사진
4	박지민	등산

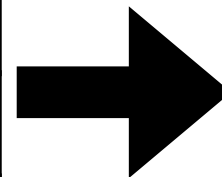
제2정규화(2NF)

제1 정규형이고 기본키가 아닌 모든 속성이 기본키에 완전 함수 종속
(부분함수종속제거)

학번	과목명	지도교수	평점
1	인공지능	이강인	A
1	파이썬	김민재	C
2	파이썬	김민재	B
3	데이터베이스	손흥민	C
4	데이터베이스	손흥민	A
4	클라우드	황희찬	D



학번	과목명	평점
1	인공지능	A
1	파이썬	C
2	파이썬	B
3	데이터베이스	C
4	데이터베이스	A
4	클라우드	D

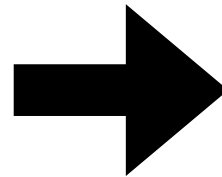


과목명	지도교수
인공지능	이강인
파이썬	김민재
데이터베이스	손흥민
클라우드	황희찬

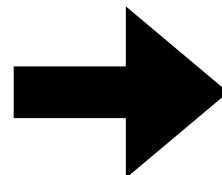
제3정규화(3NF)

제2정규형이고 이행적 함수 종속 제거

과목명	지도교수	소속
인공지능	이강인	PSG
파이썬	김민재	뮌헨
데이터베이스	손흥민	토트넘
클라우드	황희찬	울버햄튼
운영체제	김민재	뮌헨
자료구조	손흥민	토트넘



과목명	지도교수
인공지능	이강인
파이썬	김민재
데이터베이스	손흥민
클라우드	황희찬
운영체제	김민재
자료구조	손흥민



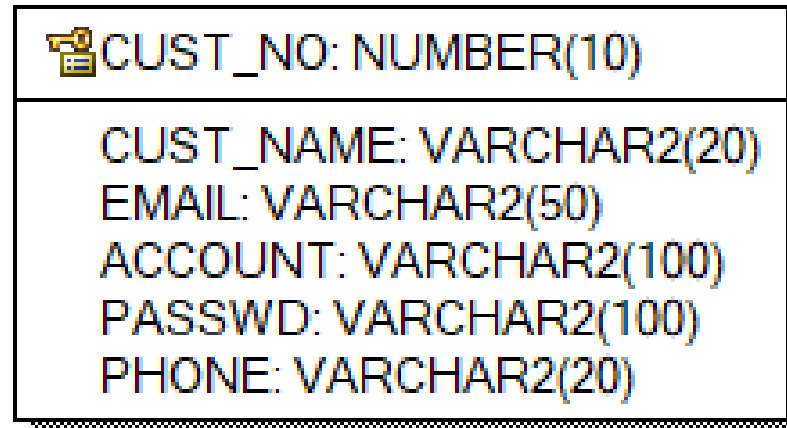
지도교수	소속
이강인	PSG
김민재	뮌헨
손흥민	토트넘
황희찬	울버햄튼

물리적 데이터 모델링

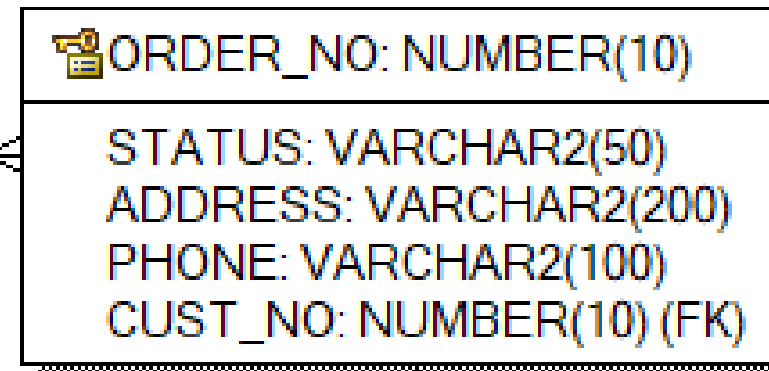
- 논리적인 데이터베이스 구조로부터 효율적이고 구현가능한 물리적 구조로 설계하는 것.
- 물리적 데이터베이스 모델링 단계
 - 개발 DBMS 선정.
 - 컬럼의 데이터 타입과 사이즈 정의
 - 데이터 사용량 분석과 사용자들의 업무 프로세스 분석
 - 역 정규화
 - 제약조건(Constraint) 정의
 - Index, View, Stored Procedure, Trigger등 Object 정의
 - Database 생성

물리적 데이터 모델링

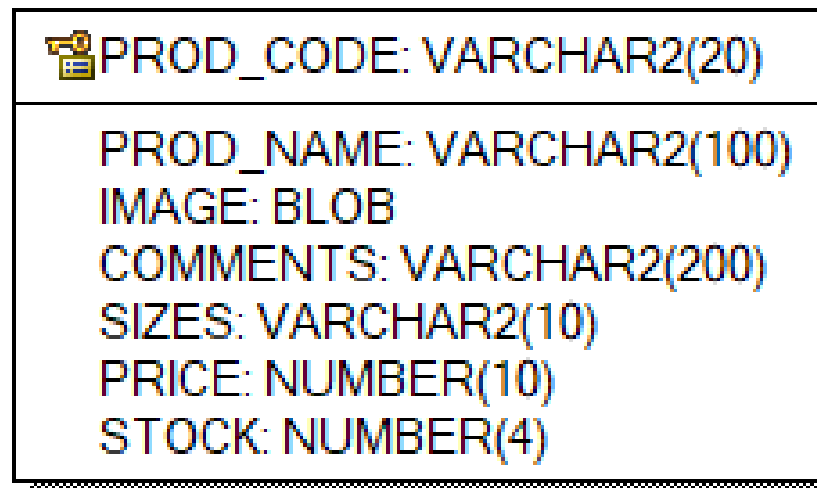
CUSTOMER



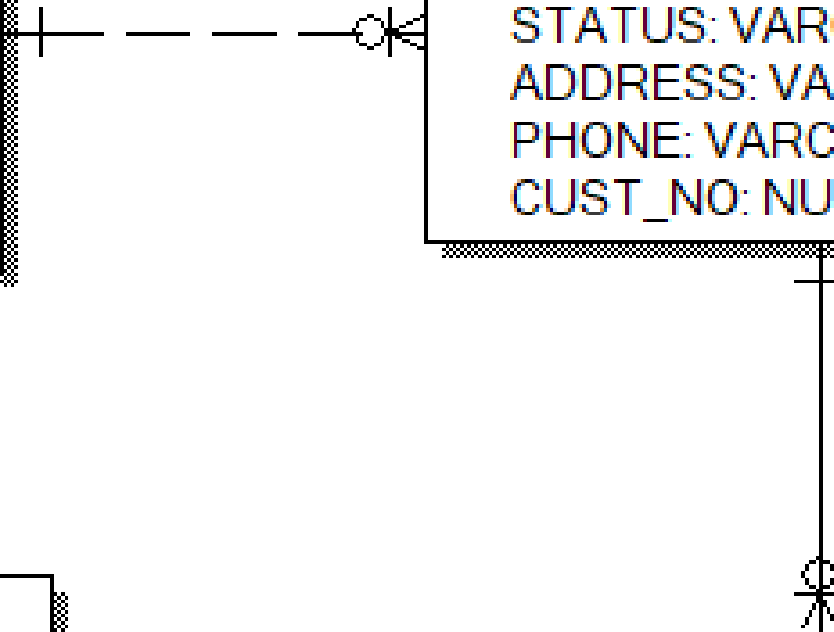
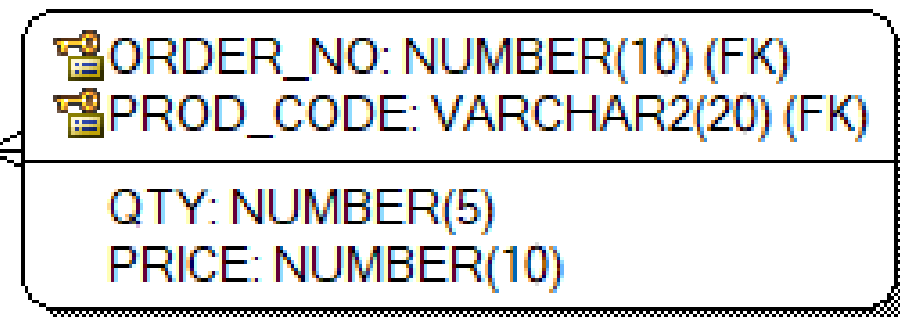
ORDERS



PRODUCTS



ORDERDETAIL



데이터베이스 설계 실습

1. 요구사항 분석
2. 요구 사항 명세서 작성
3. 명사를 개체와 속성으로 분류
4. 개체간의 관계설정
5. 물리적 설계
6. 구현

